



Xi'an Jiaotong-Liverpool University

西交利物浦大學

INT305 Machine Learning

Lecture 6

Convolutional Neural Network

卷积神经网络

Sichen Liu

Department Intelligence Science

Sichen.Liu@xjtlu.edu.cn

The problem

The problem:

semantic gap

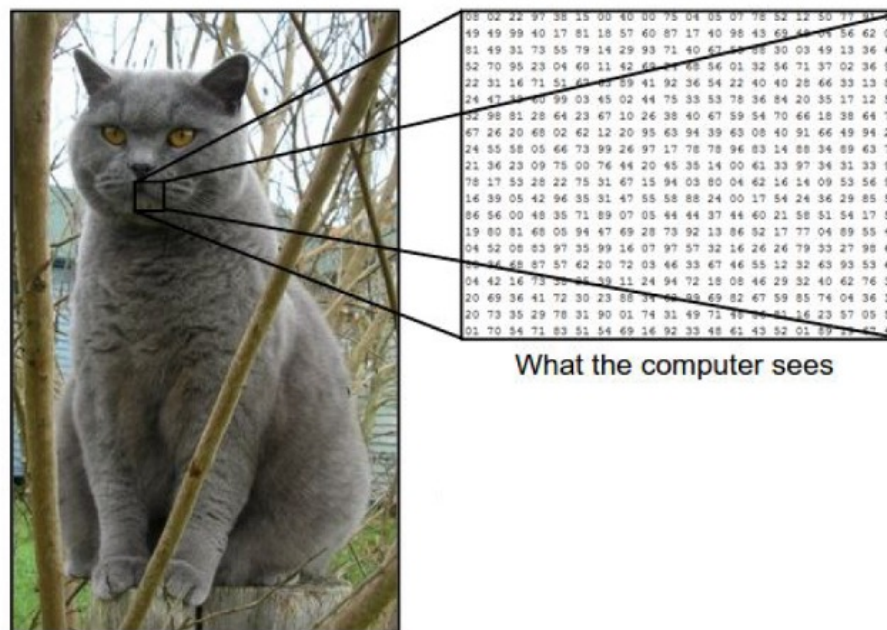
Images are represented as 3D arrays of numbers, with integers between [0, 255].

图像以三维数值数组表示，其中包含介于整数之间的数值

E.g.,

300 x 100 x 3

(3 for 3 color channels RGB)

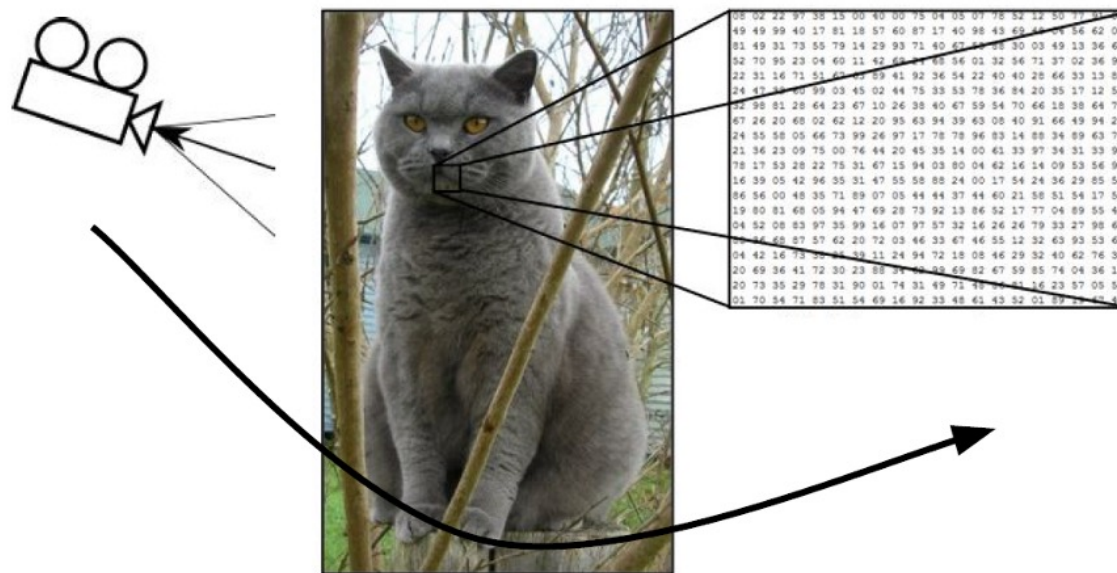


What the computer sees

一些影响分析挑战

Challenges (1)

Challenges: Viewpoint Variation 改变镜头视角



Challenges (2)

Challenges: Illumination 光照



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain

Challenges (3)

Challenges: Deformation 不正常的姿势



Challenges (4)

Challenges: Occlusion 视图中存在阻碍



Challenges (5)

Challenges: Background Clutter 和背景颜色干扰



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain

Challenges (6)

Challenges: Intraclass variation 内部类变体



An image classifier

An image classifier

```
def predict(image):  
    # ????  
    return class_label
```

Unlike e.g. sorting a list of numbers,

no obvious way to hard-code the algorithm for recognizing a cat, or other classes.

Attempts

Attempts have been made

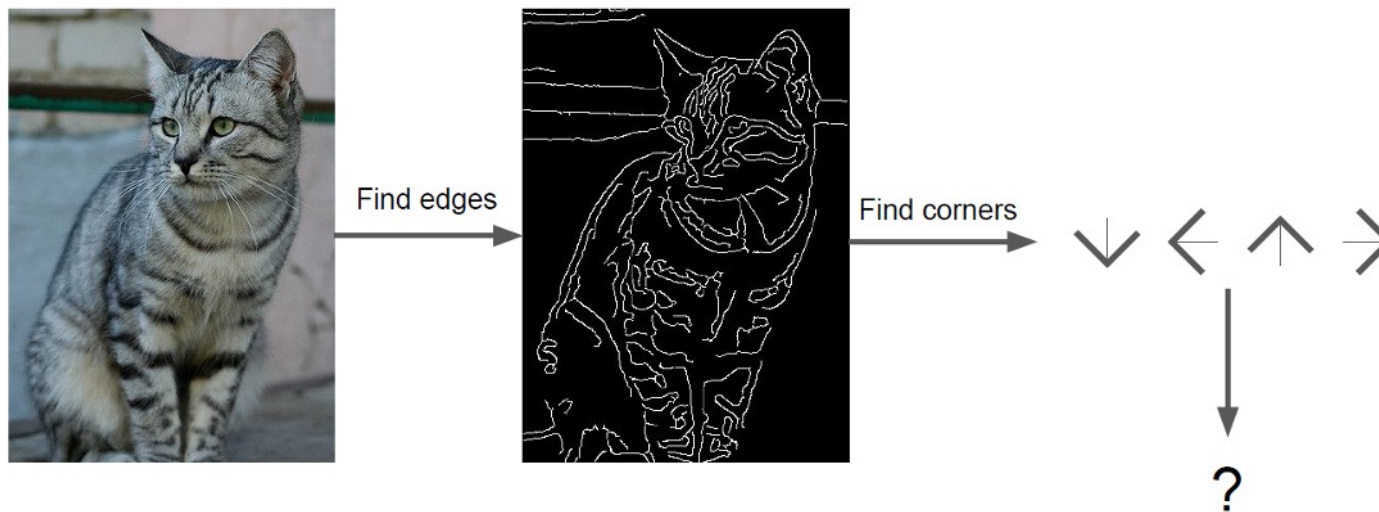


???



Attempts

Attempts have been made



Data-driven approach

Data-driven approach: 数据驱动

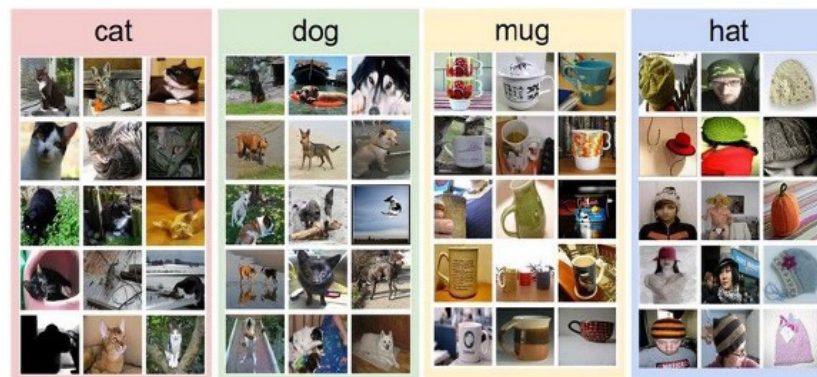
1. Collect a dataset of images and labels
2. Use Machine Learning to train an image classifier
3. Evaluate the classifier on a withheld set of test images

训练

```
def train(train_images, train_labels):  
    # build a model for images -> labels...  
    return model  
  
def predict(model, test_images):  
    # predict test_labels using the model...  
    return test_labels
```

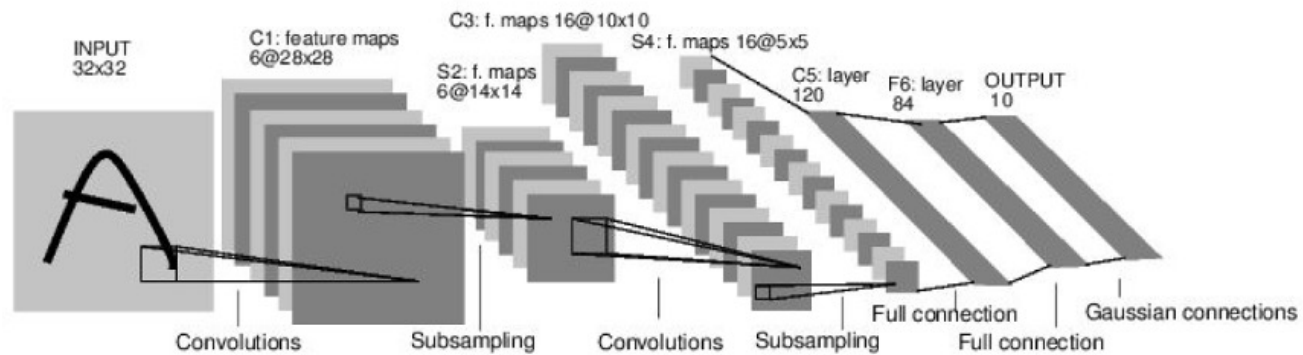
预测

Example training set



CNN

Convolutional Neural Networks

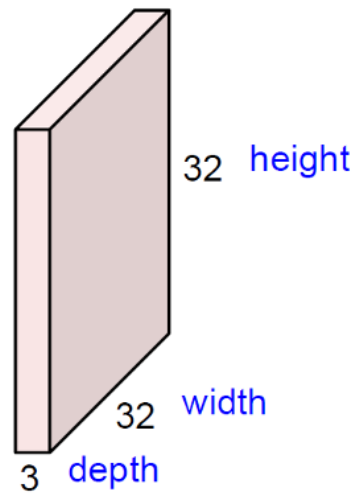


[LeNet-5, LeCun 1980]

CNN

Convolution Layer

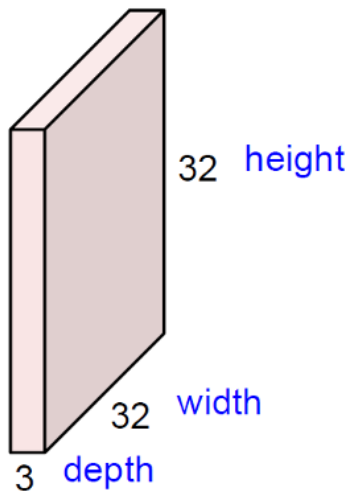
32x32x3 image



CNN

Convolution Layer

32x32x3 image



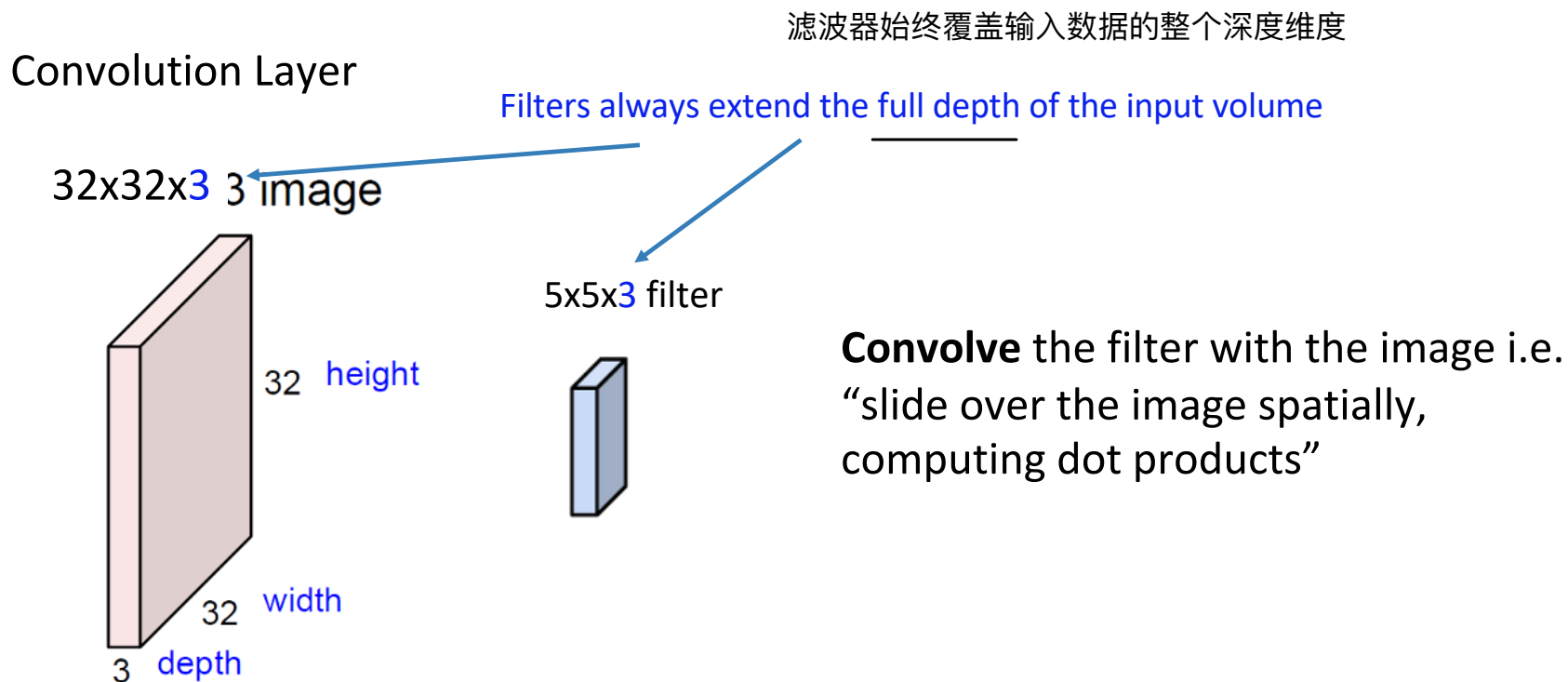
5x5x3 filter



将滤波器与图像进行卷积操作，即“在图像空间上滑动滤波器，逐点计算内积”

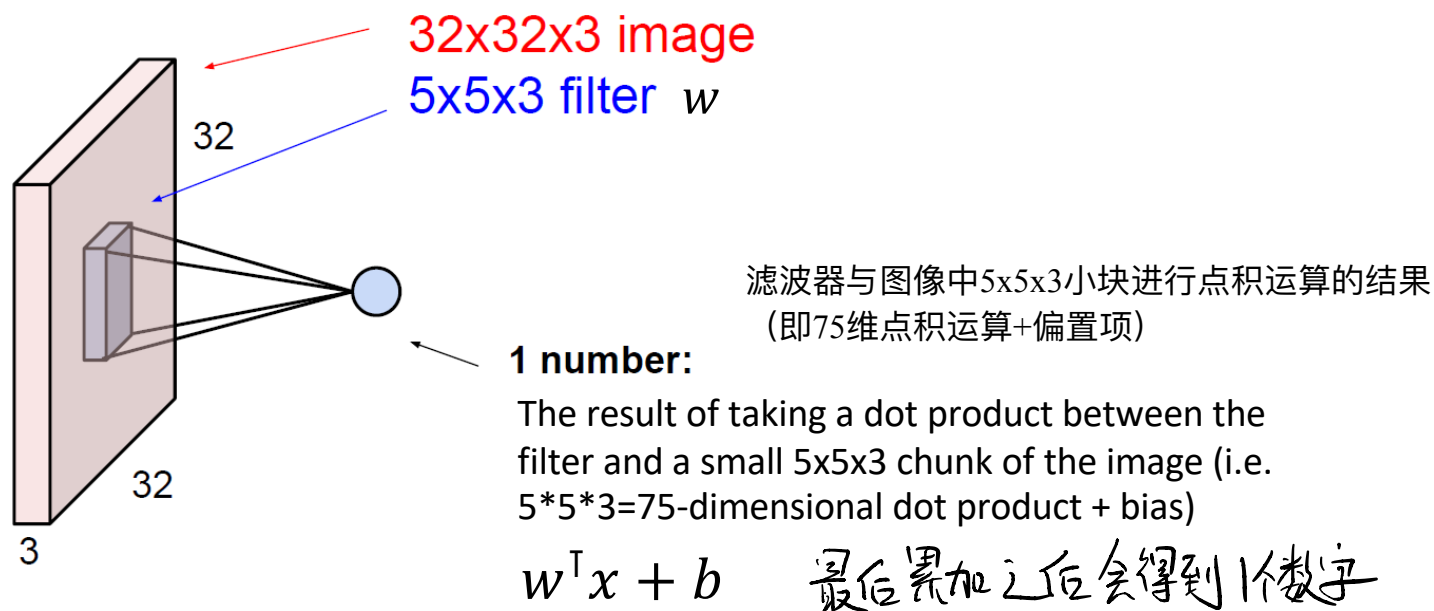
Convolve the filter with the image i.e.
“slide over the image spatially,
computing dot products”

CNN



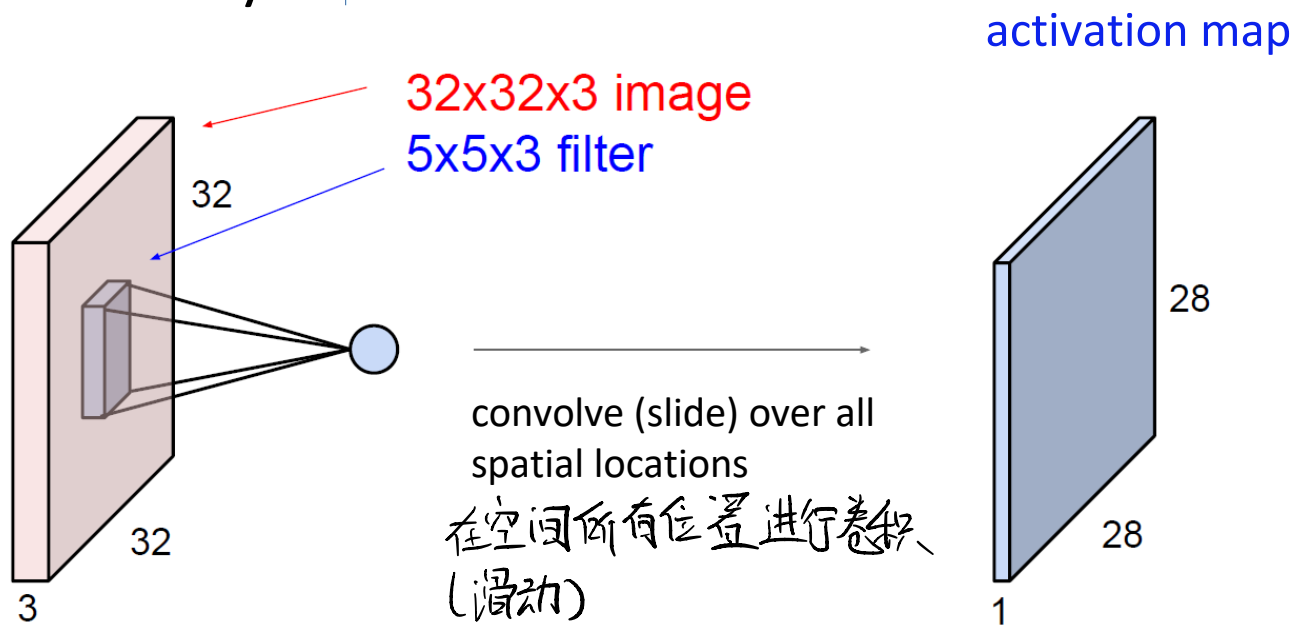
CNN

Convolution Layer



CNN

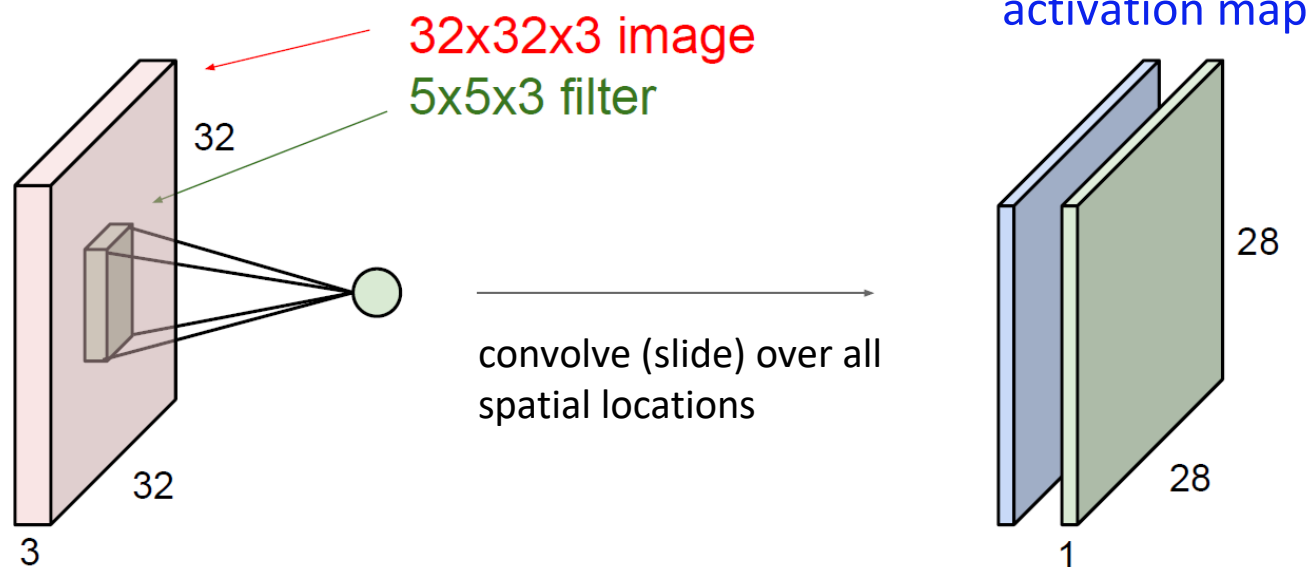
Convolution Layer



CNN

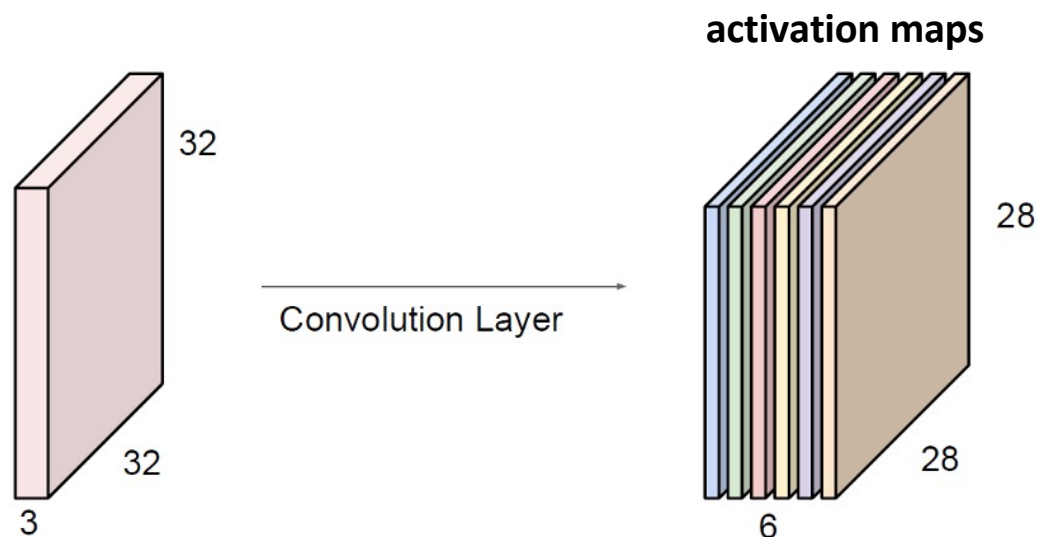
Convolution Layer

consider a second, **green** filter



CNN

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



有几个滤波器
最终得到对应数量
的 activation maps.

We stack these up to get a “new image” of size 28x28x6!

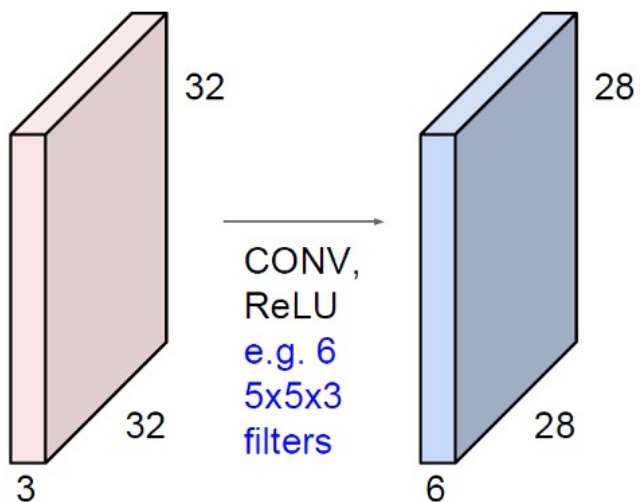
大小: $(\frac{32-5}{1}) + 1 = 28$

★ $(\frac{\text{image size} - \text{filter size}}{\text{stride}}) + 1$

CNN

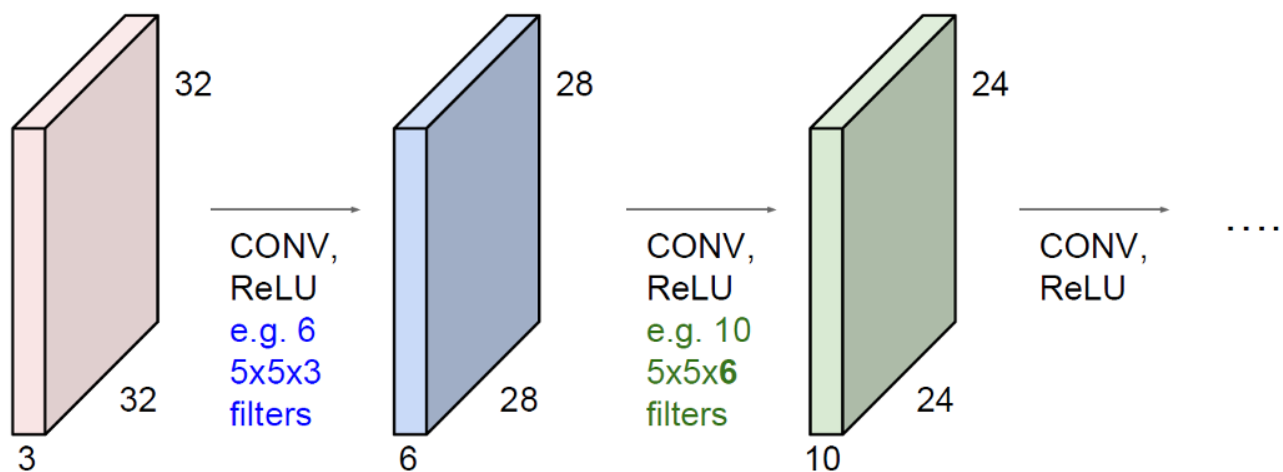
Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions

预览：ConvNet是由卷积层序列组成，其间穿插激活函数的结构。



CNN

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions

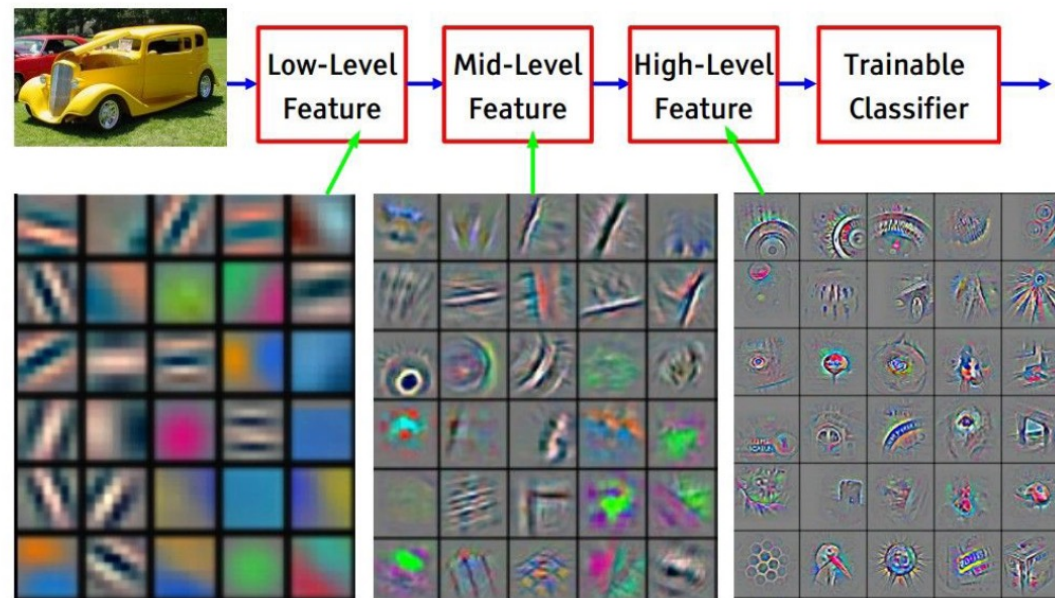


比如 conv 之后得到 $\begin{bmatrix} 2 & -1 \\ -3 & 5 \end{bmatrix}$, 然后执行 ReLU $\max(0, x)$, 将得到的 feature 转变为 $\begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix}$
然后交给 max pooling 取其中的最大值为 5。

CNN

Preview

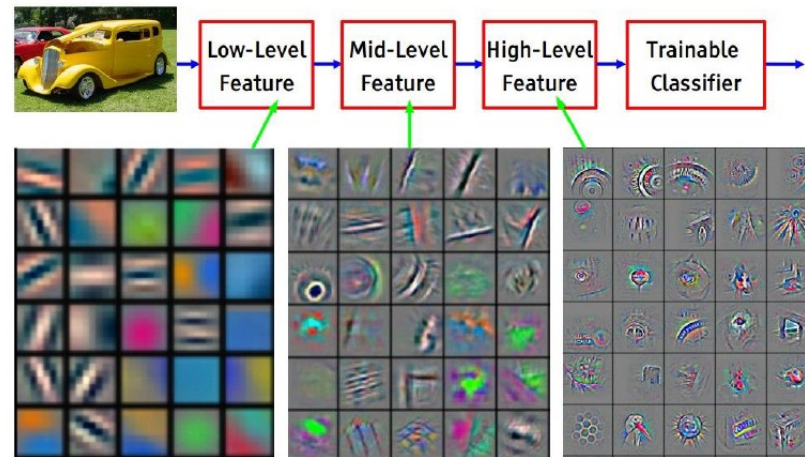
*[From recent Yann
LeCun slides]*



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

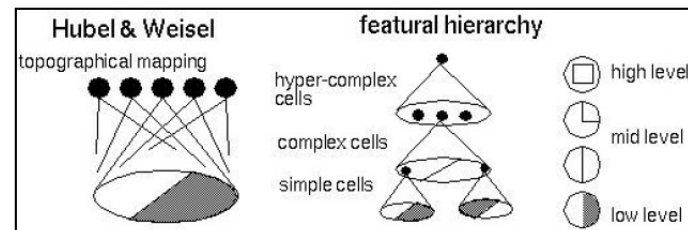
CNN

Preview



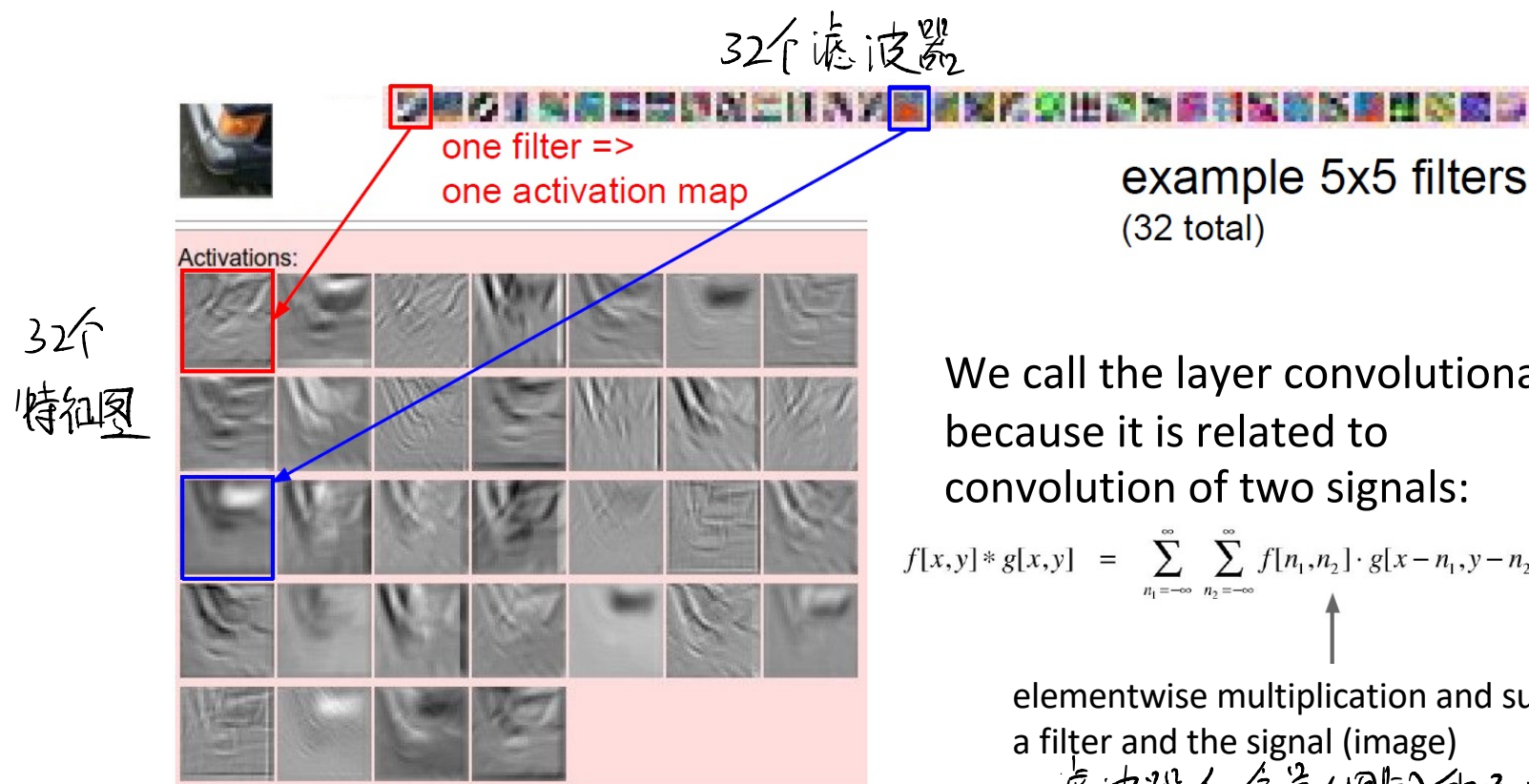
[From recent Yann LeCun slides]

Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]



CNN

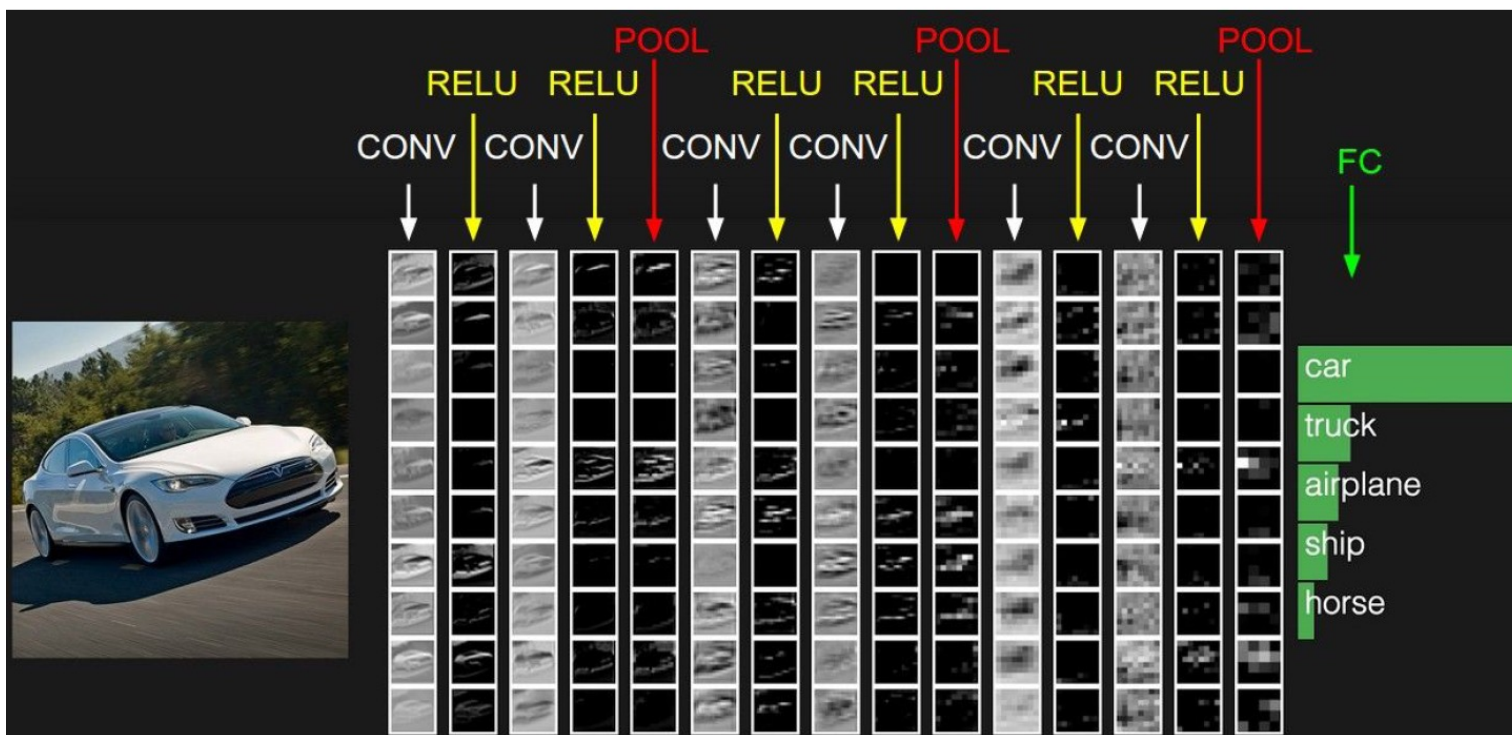
一个滤波器 $\Leftrightarrow$ 一个激活图



滤波器和信号(图片)的逐元素相乘然后求和

CNN

Preview:

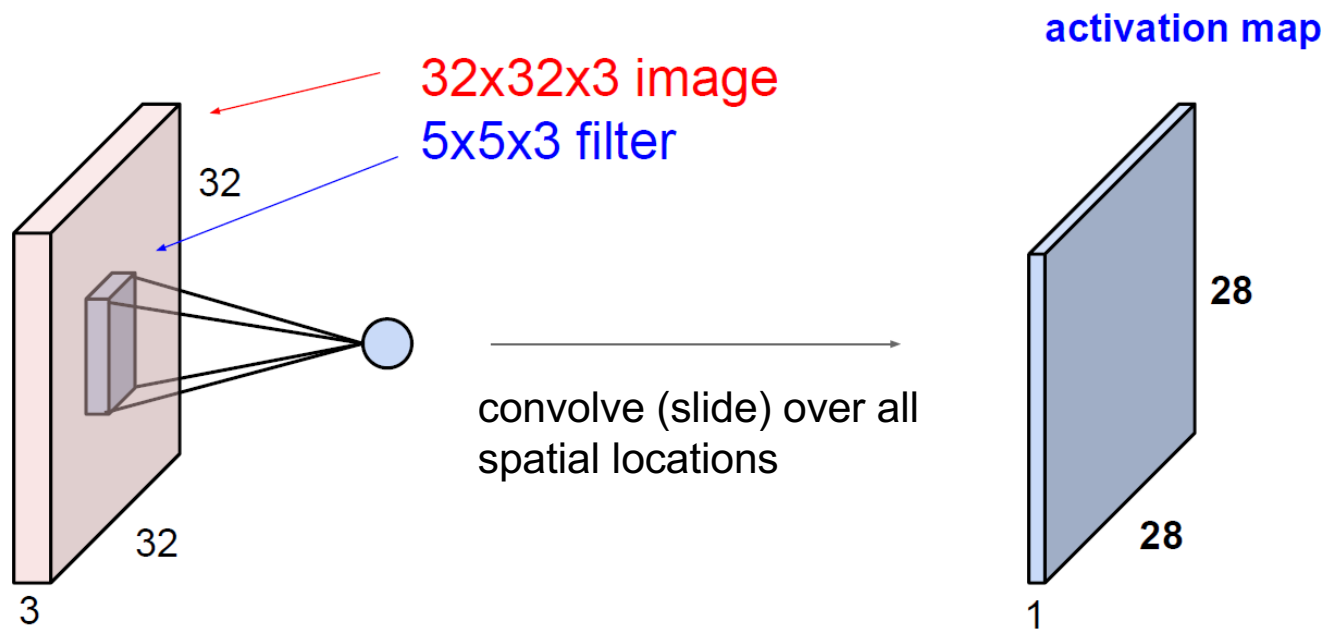


CONV -> RELU: 在上一步被浓缩过的特征图上，继续寻找更高级、更抽象的特征组合（比如“车灯+格栅”的组合）。

也 POOL: 再次进行“总结和压缩”，特征图变得更小。

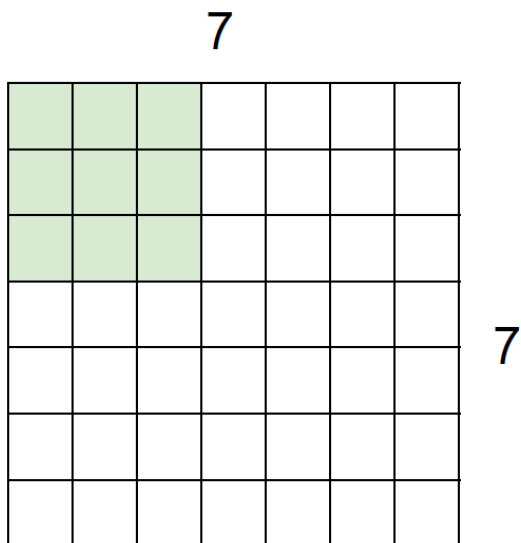
CNN

A closer look at spatial dimensions:



CNN

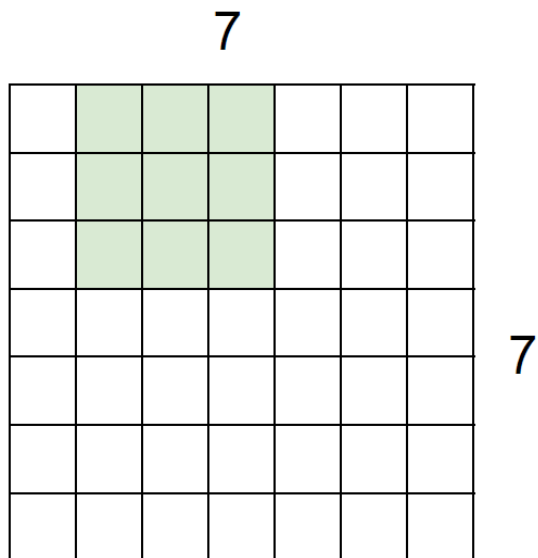
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

CNN

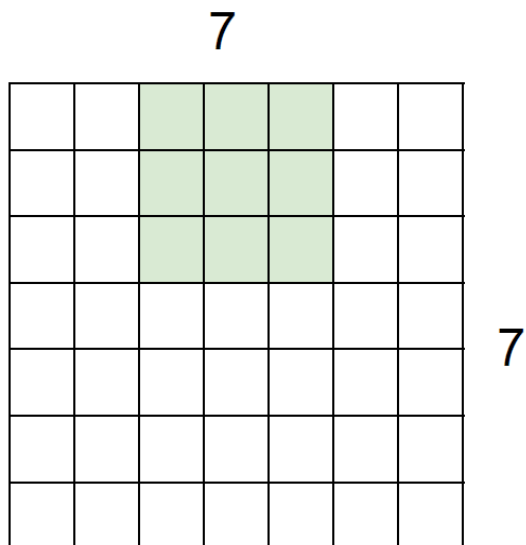
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

CNN

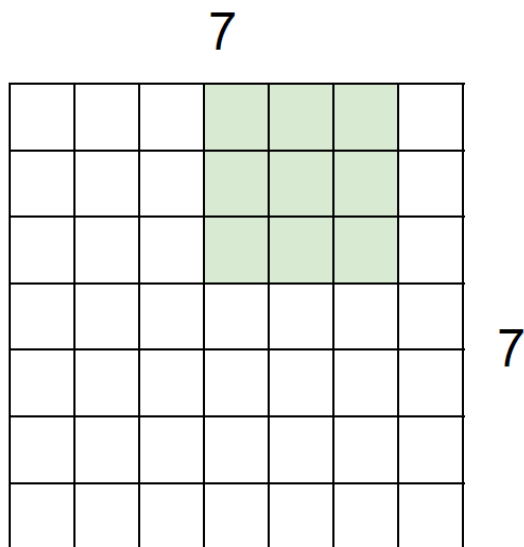
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

CNN

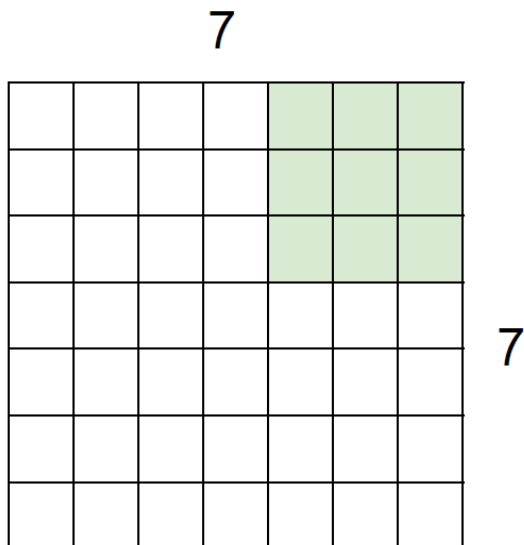
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

CNN

A closer look at spatial dimensions:

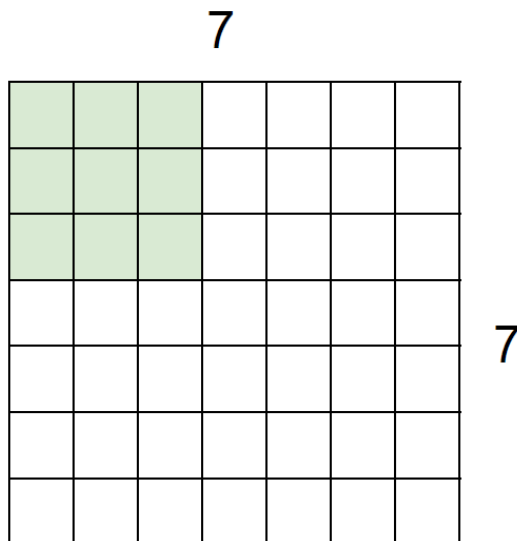


7x7 input (spatially)
assume 3x3 filter

=> **5x5 output**

CNN

A closer look at spatial dimensions:



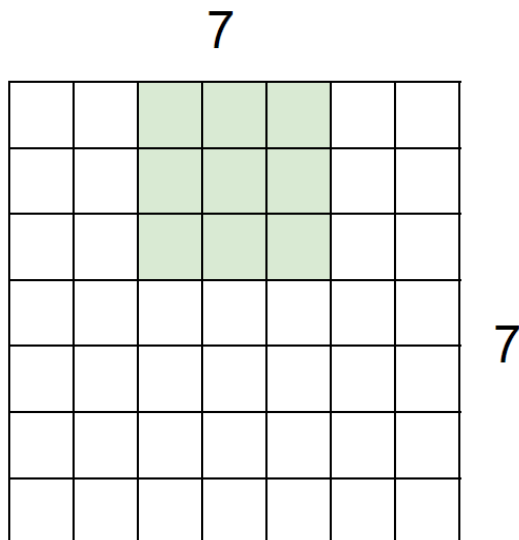
7x7 input (spatially)

assume 3x3 filter

applied **with stride 2** 步长为2

CNN

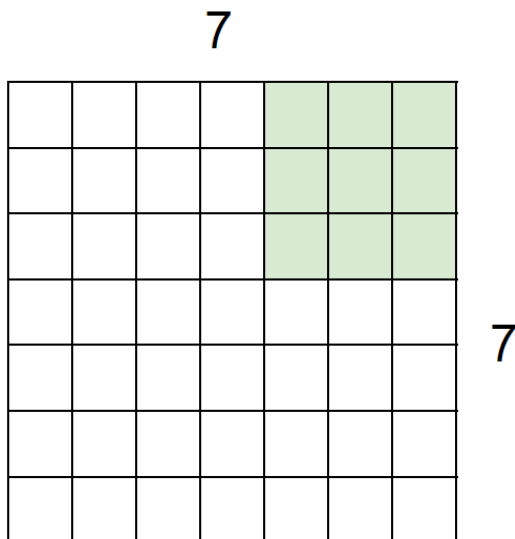
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

CNN

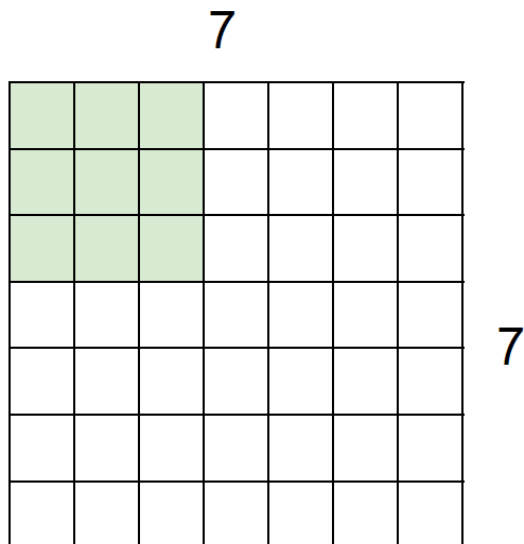
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

CNN

A closer look at spatial dimensions:



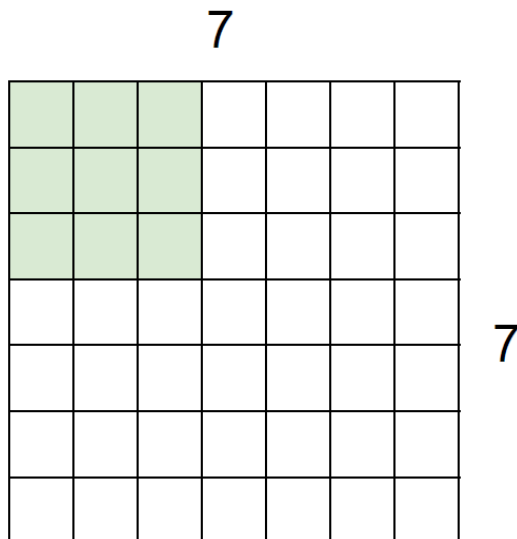
7x7 input (spatially)

assume 3x3 filter

applied **with stride 3?** 歩幅

CNN

A closer look at spatial dimensions:

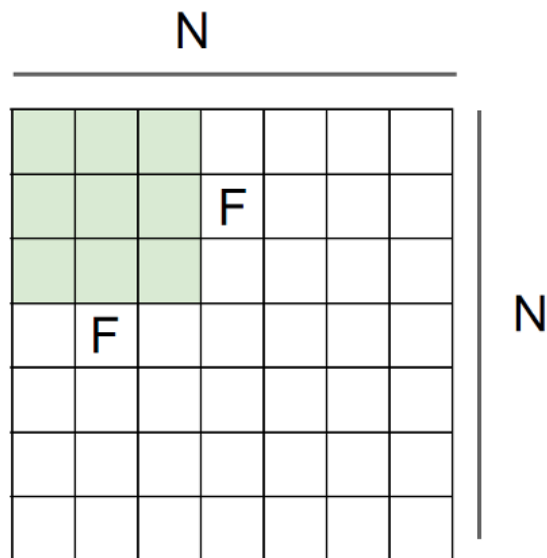


7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!

Cannot apply 3x3 filter on 7x7
input with stride 3.

CNN



Output size:
 $(N - F)/\text{stride} + 1$

e.g., $N=7, F=3$:

stride 1 $\Rightarrow (7 - 3)/1 + 1 = 5$

stride 2 $\Rightarrow (7 - 3)/2 + 1 = 3$

stride 3 $\Rightarrow (7 - 3)/3 + 1 = 2.33 \dots$

不能整除

CNN

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g., input 7x7

3x3 filter, applied with **stride 1** 步长为1

pad with 1 pixel border => what is the output?

边缘外加一圈像素

(recall:)

$(N - F) / \text{stride} + 1$

CNN

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g., input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

CNN

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g., input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

zero padding 通常取值

e.g., $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

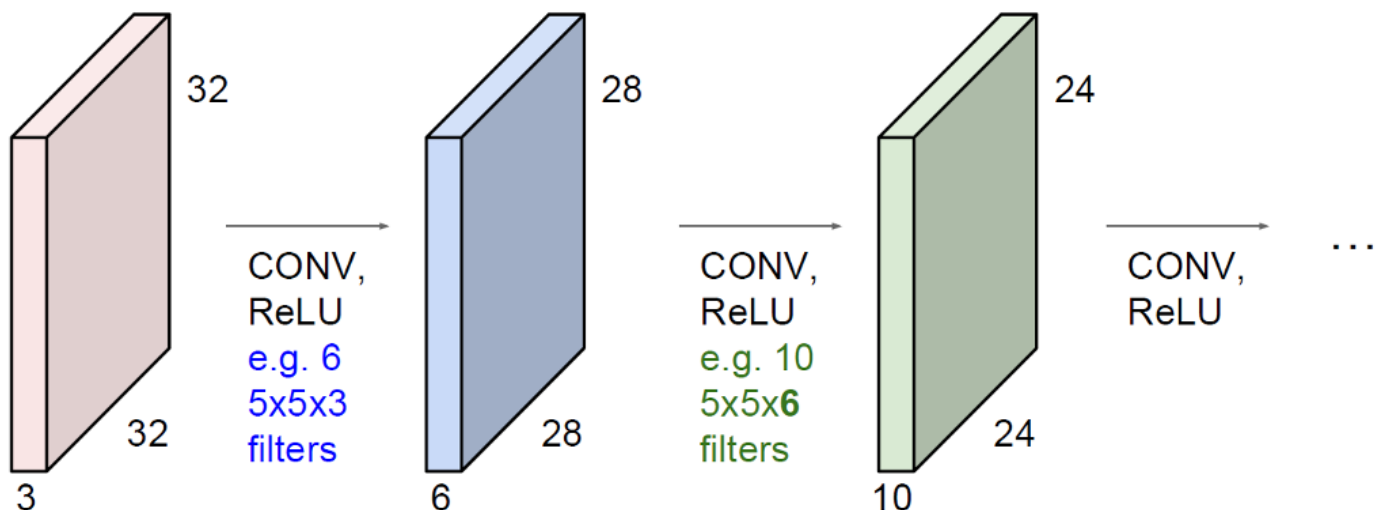
$F = 7 \Rightarrow$ zero pad with 3

CNN

例如，32x32的输入通过5x5滤波器重复卷积会导致空间维度快速收缩（32→28→24...）。收缩速度过快效果不佳，会影响模型性能。

Remember back to ...

E.g. 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially! (32 -> 28 -> 24 ...). Shrinking too fast is not good, doesn't work well.



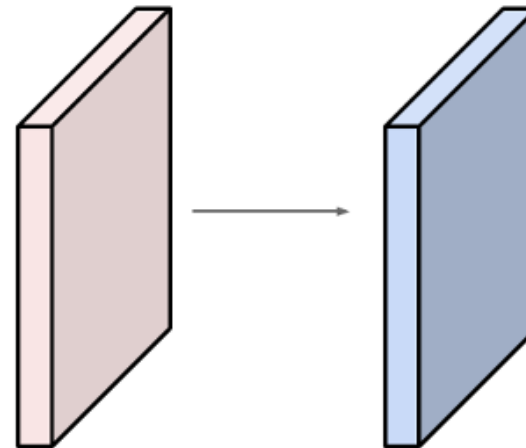
CNN

Examples time:

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

Output volume size:?



CNN



Examples time:

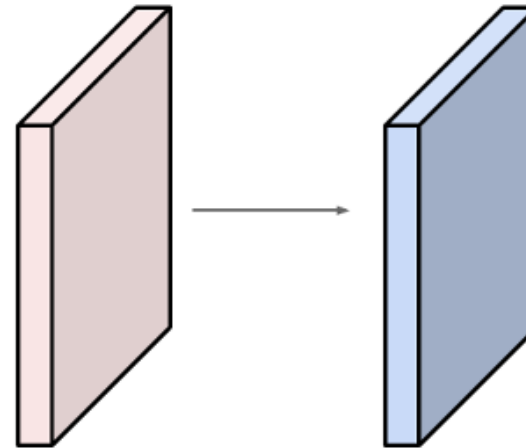
Input volume: **32x32x3**

10 **5x5** filters with stride **1**, pad **2**

Output volume size:

$(32 + 2 * 2 - 5) / 1 + 1 = 32$ spatially, so

32x32x10



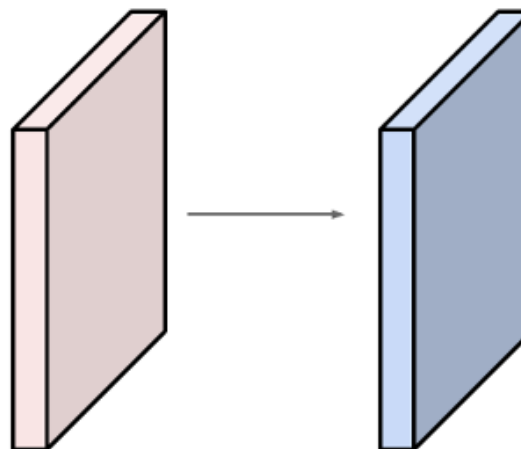
CNN

Examples time:

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?



CNN

★ Examples time:

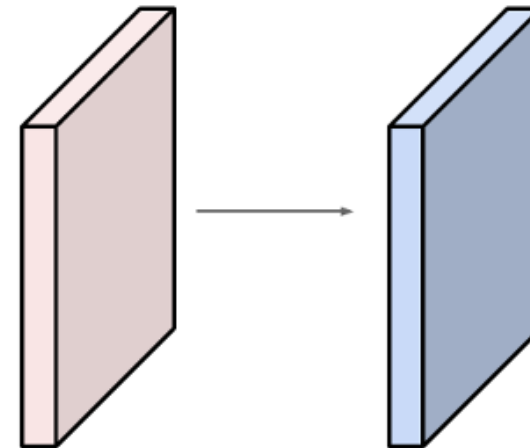
Input volume: $32 \times 32 \times 3$

10 5×5 filters with stride 1, pad 2

Number of parameters in this layer?

Each filter has $5 * 5 * 3 + 1 = 76$ params

=> $76 * 10 = 760$



(+1 for bias)

CNN

★ **Summary.** To summarize, the Conv layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$

width, height, depth

- Requires four hyperparameters:

- Number of filters K , 滤波器数量
- Their spatial extent F , 滤波器的长宽大小
- The stride S , 步长
- The amount of zero padding P . padding 的层数

拿一个的滤波器（比如“边缘探测器”），在扫描整张图片时，用的是同一组权重。它不会在左上角用一套权重，到右下角又换一套。

- Produces a volume of size $W_2 \times H_2 \times D_2$ where:

- $W_2 = (W_1 - F + 2P)/S + 1$
- $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
- $D_2 = K$ 输出深度 = 滤波器数量

这就是“参数共享”：一个滤波器（一组参数）被共享给图片的所有位置使用。

- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d -th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d -th filter over the input volume with a stride of S , and then offset by d -th bias.

输出方块中的第 d 个切片 (D -th depth slice)，就是用第 d 个滤波器 (d -th filter) 在整个输入方块上做完卷积（滑动、乘加）并加上偏置项 d -th bias 后的结果。

CNN

Summary. To summarize, the Conv layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
 - ❑ Number of filters K ,
 - ❑ Their spatial extent F ,
 - ❑ The stride S ,
 - ❑ The amount of zero padding P .
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - ❑ $W_2 = (W_1 - F + 2P)/S + 1$
 - ❑ $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
 - ❑ $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d -th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d -th filter over the input volume with a stride of S , and then offset by d -th bias.

Common settings:

K = (powers of 2, e.g., 32, 64, 128, 512)

- $F = 3, S = 1, P = 1$

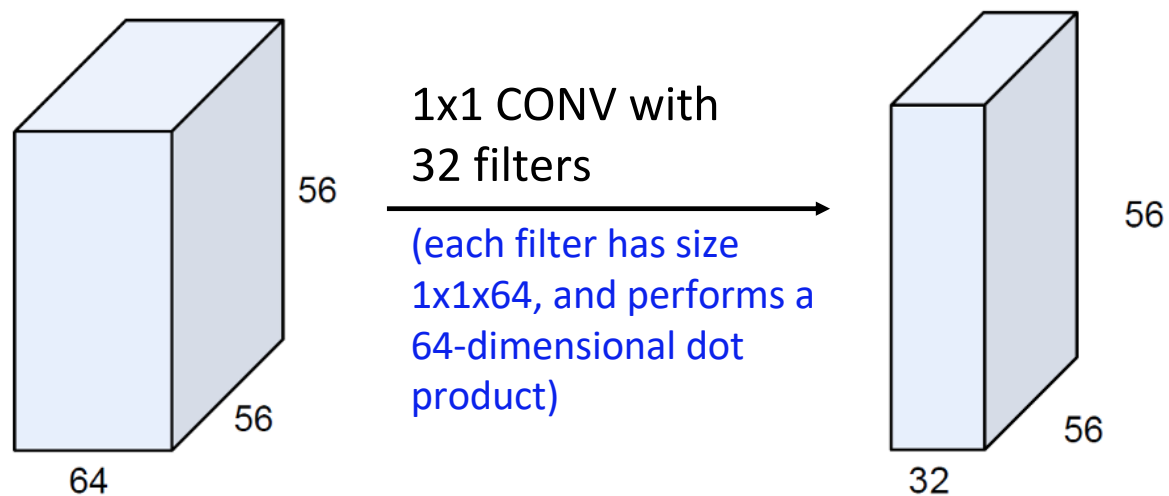
- $F = 5, S = 1, P = 2$

- $F = 5, S = 2, P = ?$ (whatever fits)

- $F = 1, S = 1, P = 0$

CNN

(btw, 1x1 convolution layers make perfect sense)



CNN

Example: CONV layer in Torch

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
 - Number of filters K ,
 - their spatial extent F ,
 - the stride S ,
 - the amount of zero padding P .

SpatialConvolution

```
module = nn.SpatialConvolution(nInputPlane, nOutputPlane, kW, kH, [dW], [dH], [padW], [padH])
```

Applies a 2D convolution over an input image composed of several input planes. The `input` tensor in `forward(input)` is expected to be a 3D tensor (`nInputPlane` x `height` x `width`).

The parameters are the following:

- `nInputPlane` : The number of expected input planes in the image given into `forward()`.
- `nOutputPlane` : The number of output planes the convolution layer will produce.
- `kW` : The kernel width of the convolution
- `kH` : The kernel height of the convolution
- `dW` : The step of the convolution in the width dimension. Default is `1`.
- `dH` : The step of the convolution in the height dimension. Default is `1`.
- `padW` : The additional zeros added per width to the input planes. Default is `0`, a good number is $(kW-1)/2$.
- `padH` : The additional zeros added per height to the input planes. Default is `padW`, a good number is $(kH-1)/2$.

Note that depending of the size of your kernel, several (of the last) columns or rows of the input image might be lost. It is up to the user to add proper padding in images.

If the input image is a 3D tensor `nInputPlane` x `height` x `width`, the output image size will be `nOutputPlane` x `oheight` x `owidth` where

```
owidth = floor((width + 2*padW - kW) / dW + 1)
oheight = floor((height + 2*padH - kH) / dH + 1)
```

CNN

Example: CONV layer in Caffe

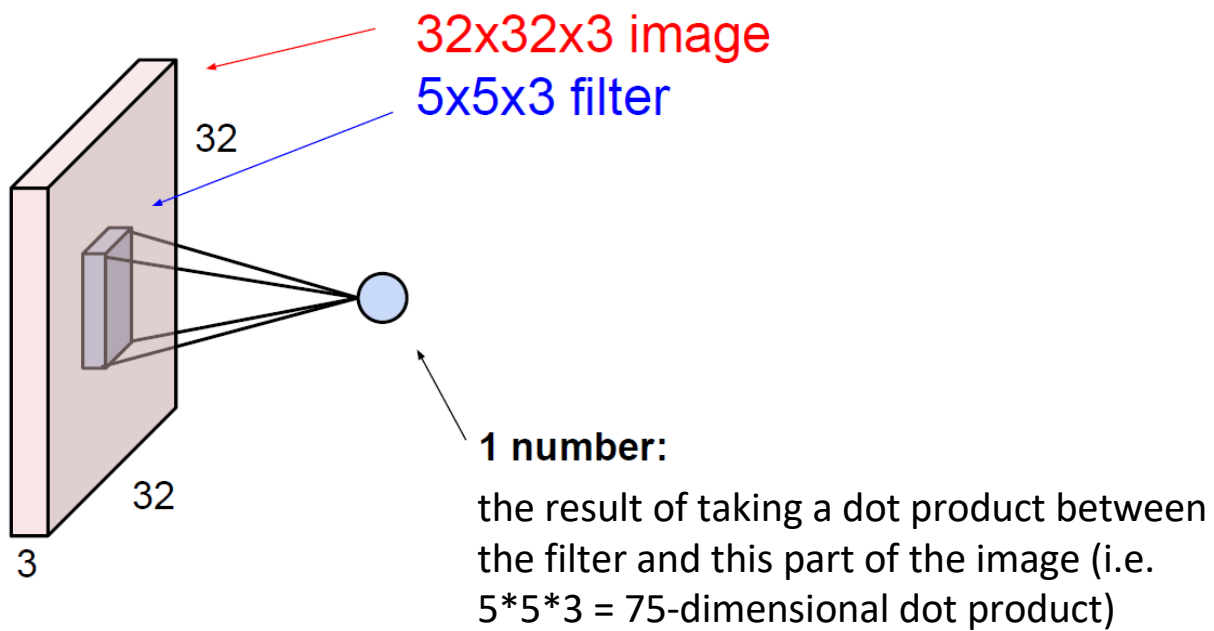
Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
 - Number of filters K ,
 - their spatial extent F ,
 - the stride S ,
 - the amount of zero padding P .

```
layer {
  name: "conv1"
  type: "Convolution"
  bottom: "data"
  top: "conv1"
  # learning rate and decay multipliers for the filters
  param { lr_mult: 1 decay_mult: 1 }
  # learning rate and decay multipliers for the biases
  param { lr_mult: 2 decay_mult: 0 }
  convolution_param {
    num_output: 96      # learn 96 filters
    kernel_size: 11     # each filter is 11x11
    stride: 4           # step 4 pixels between each filter application
    weight_filler {
      type: "gaussian" # initialize the filters from a Gaussian
      std: 0.01        # distribution with stdev 0.01 (default mean: 0)
    }
    bias_filler {
      type: "constant" # initialize the biases to zero (0)
      value: 0
    }
  }
}
```

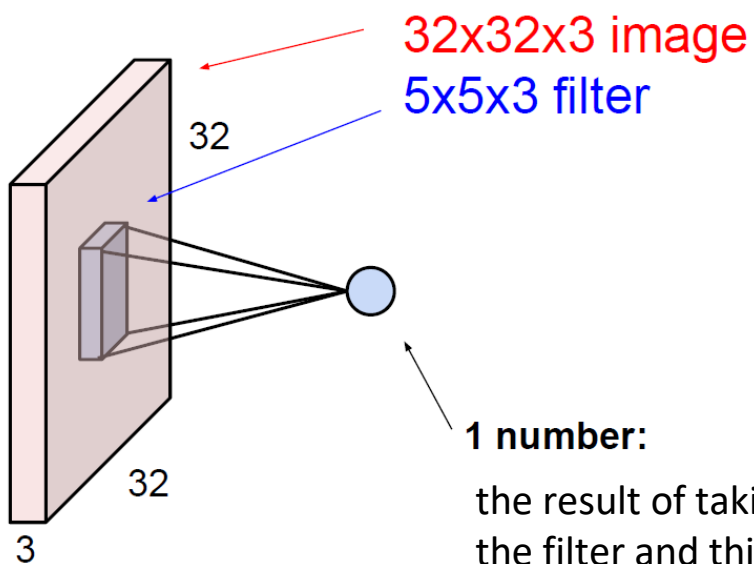
CNN

The brain/neuron view of CONV Layer



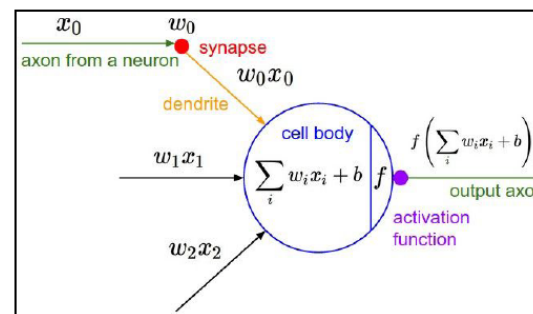
CNN

The brain/neuron view of CONV Layer



1 number:

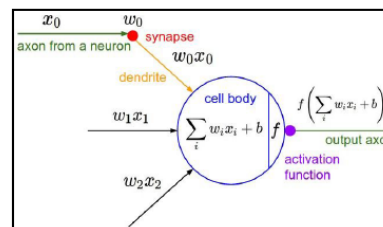
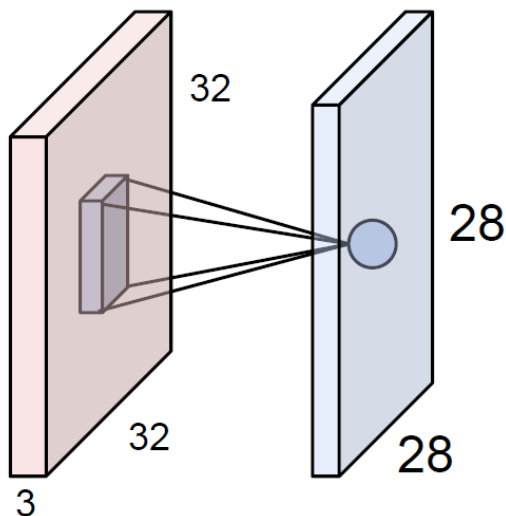
the result of taking a dot product between
the filter and this part of the image (i.e.
 $5 \times 5 \times 3 = 75$ -dimensional dot product)



It's just a neuron with
local connectivity...

CNN

The brain/neuron view of CONV Layer



An activation map is a 28x28 sheet of neuron outputs:

1. Each is connected to a small region in the input
2. All of them share parameters

“5x5 filter” -> “5x5 receptive field for each neuron”

CNN只关注局部:

32x32x3: 输入

5x5x3: 神经元的权重

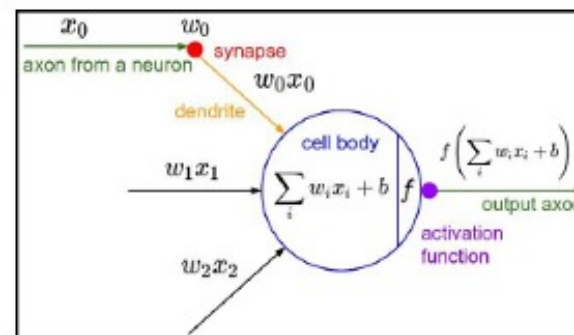
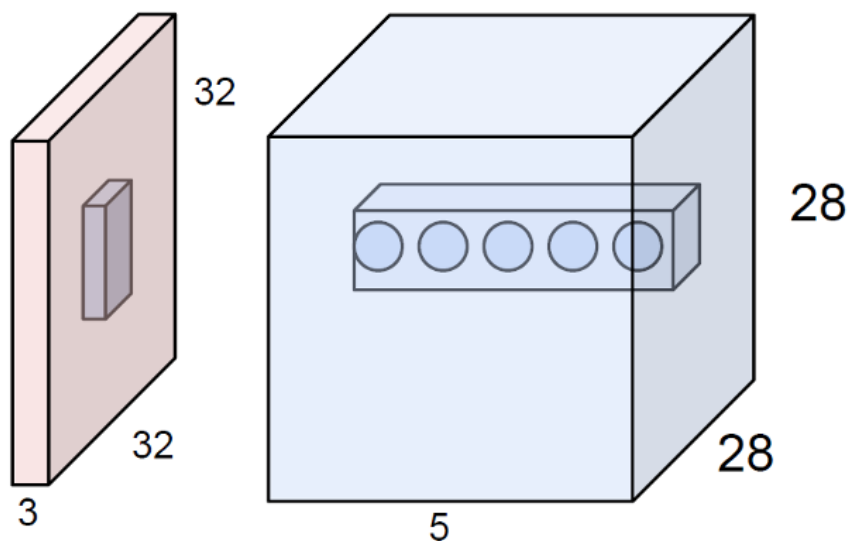
总共的权重: 5x5x3 = 75个权重

相比于传统的全连接 FC layer
因为每个神经元会连接到前一层的所有输入, 在这里输入是 32x32x3 = 3072,
那么 FC layer 就需要 3072个权重.

CNN中, 滤波器从左到右
从上到下滑动中, 实际上在
不停重复使用同一组75个
权重.

CNN

The brain/neuron view of CONV Layer



E.g., with 5 filters, CONV layer consists of neurons arranged in a 3D grid (28x28x5)

There will be 5 different neurons all looking at the same region in the input volume

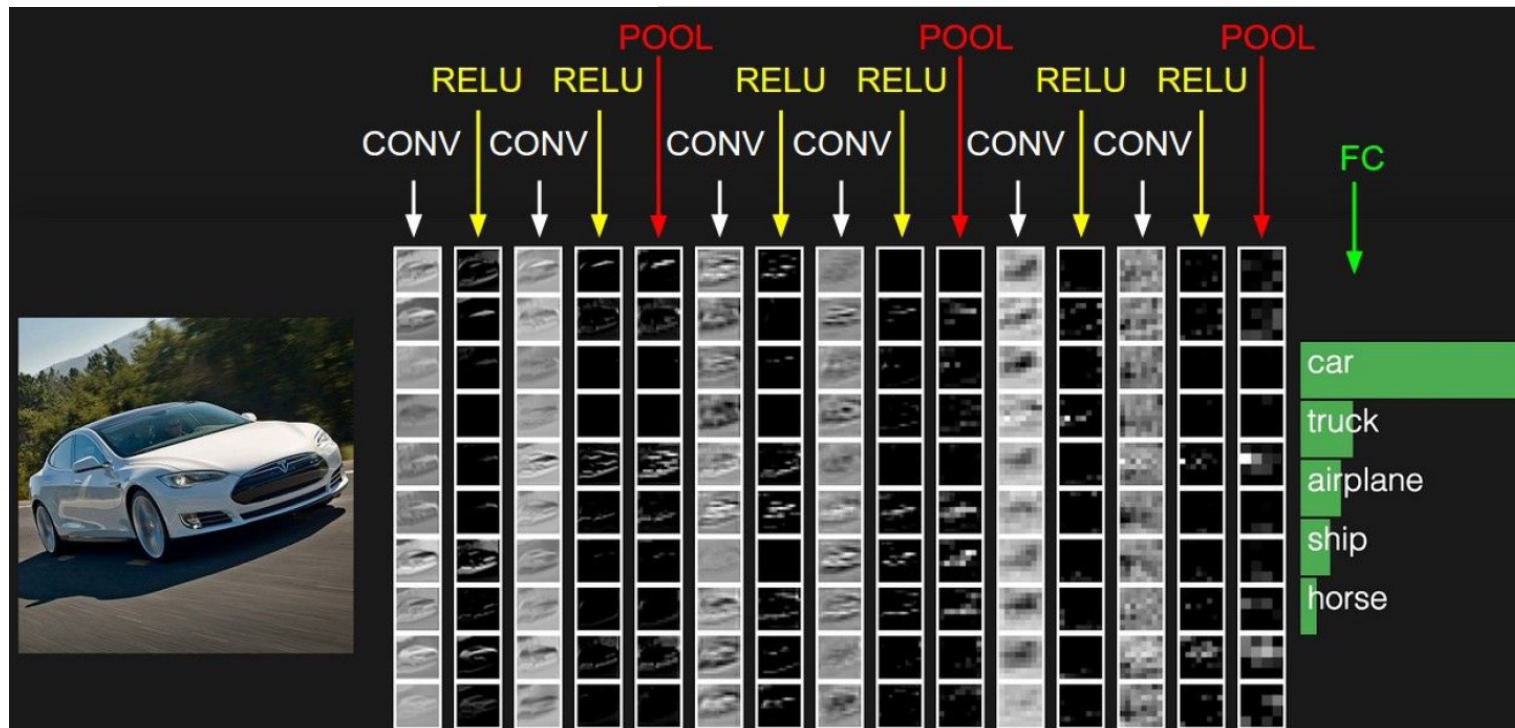
一个卷积层 (CONV Layer) 就是一大群“克隆”神经元的集合。

"克隆"体现在哪里? (参数共享): 滤波器 1 在 (x,y) 位置的那个神经元, 和它在 (x+1, y) 位置的那个神经元, 是“克隆”的 (共享同一组权重)。

"不同"体现在哪里? (多滤波器): 在同一个 (x,y) 位置, 滤波器 1 的神经元和 滤波器 2 的神经元是不同的 (它们权重不同, 在寻找不同的特征)。

CNN

two more layers to go: POOL/FC

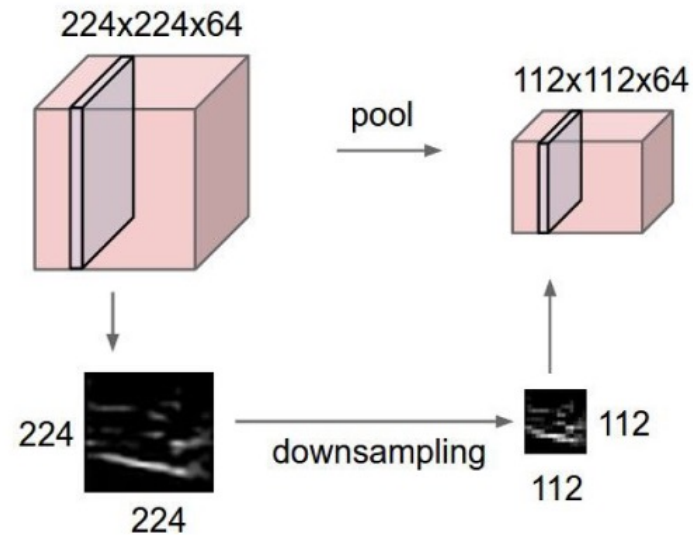


CNN

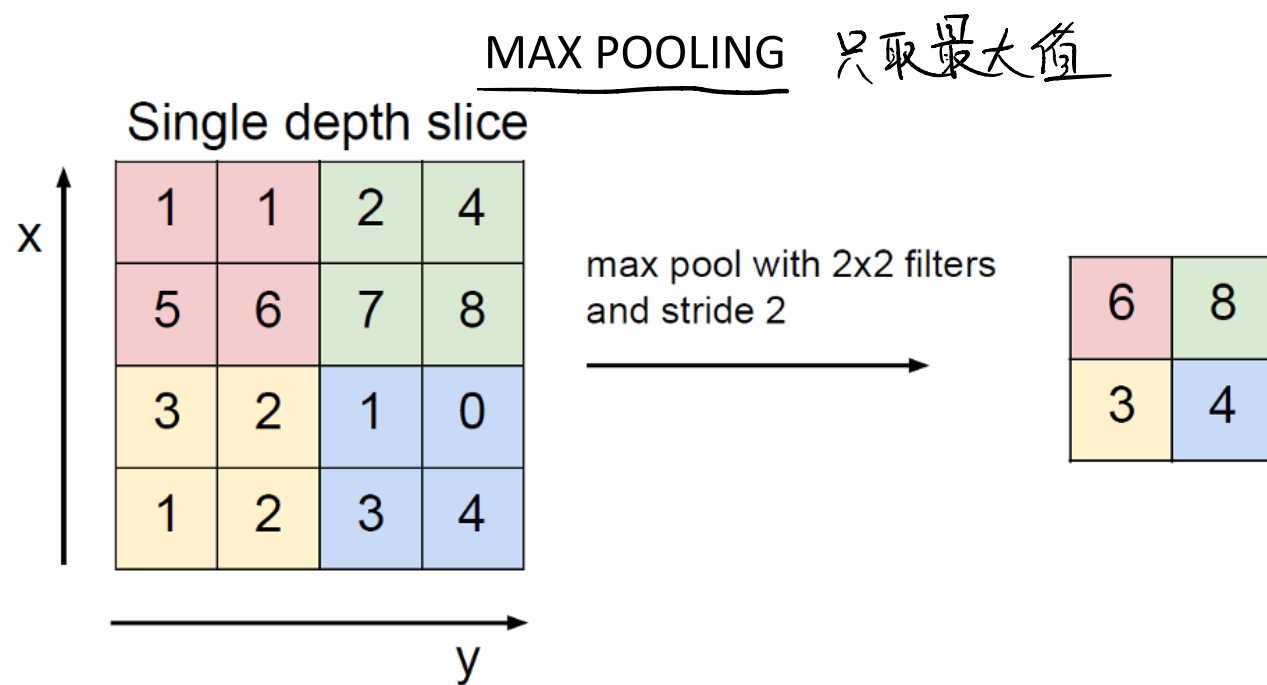
Pooling layer

池化层使特征表示更小且更易于处理，并对每个激活图独立进行运算。

- makes the representations smaller and more manageable
- operates over each activation map independently:



CNN



CNN

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
 - ▣ Their spatial extent F , 空间尺寸
 - ▣ The stride S , 步长
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $\left\{ \begin{array}{l} \text{▣ } W_2 = (W_1 - F)/S + 1 \\ \text{▣ } H_2 = (H_1 - F)/S + 1 \\ \text{▣ } D_2 = D_1 \end{array} \right.$
- Introduces zero parameters since it computes a fixed function of the input
- Note that it is not common to use zero-padding for Pooling layers

CNN

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
 - ▣ Their spatial extent F ,
 - ▣ The stride S ,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:

- ▣ $W_2 = (W_1 - F) / S + 1$
- ▣ $H_2 = (H_1 - F) / S + 1$
- ▣ $D_2 = D_1$

Common settings:

$$F = 2, S = 2$$

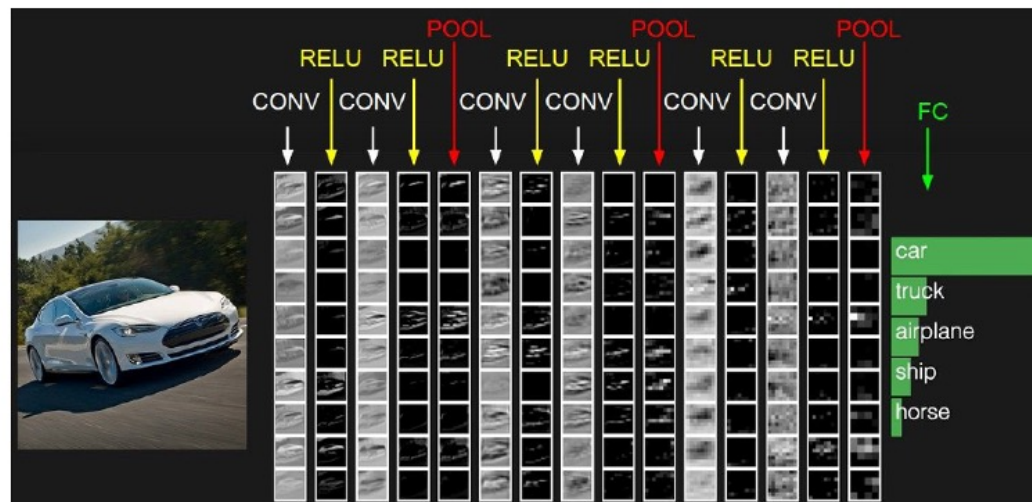
$$F = 3, S = 2$$

- Introduces zero parameters since it computes a fixed function of the input
- Note that it is not common to use zero-padding for Pooling layers

CNN

Fully Connected Layer (FC layer)

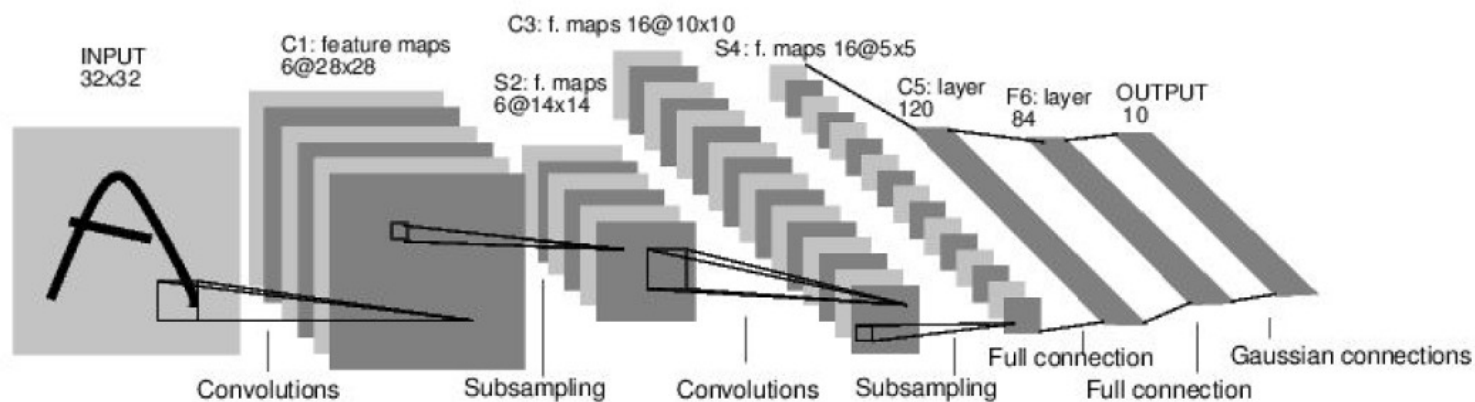
- Contains neurons that connect to the entire input volume, as in ordinary Neural Networks



CNN

Case Study: LeNet-5

[LeCun et al., 1998]



Conv filters were 5x5, applied at strid 1

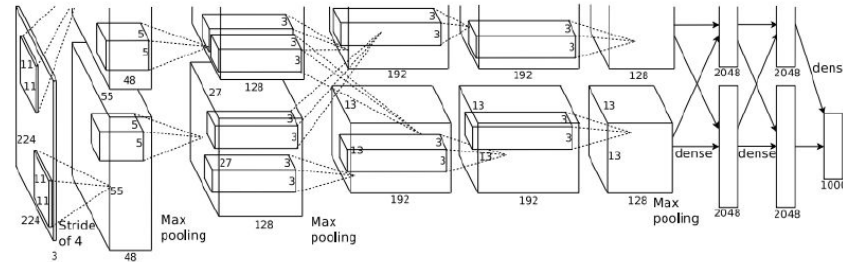
Subsampling (Pooling) layers were 2x2 applied at stride 2

i.e. architecture is [CONV-POOL-CONV-POOL-CONV-FC]

CNN

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3

First layer (CONV1): 96 11x11 filters applied at stride 4

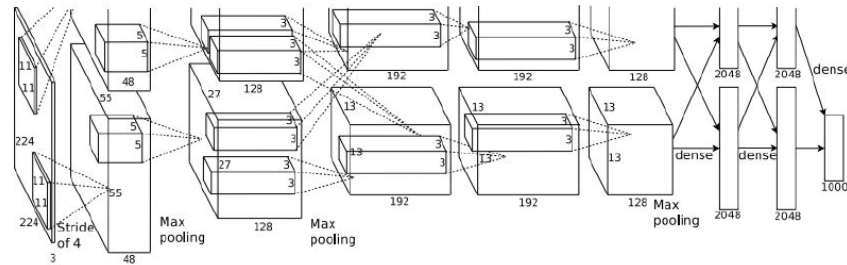
=>

Q: what is the output volume size? Hint: $(227-11)/4+1=55$

CNN

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3

First layer (CONV1): 96 11x11 filters applied at stride 4

=>

Output volume [55x55x96]

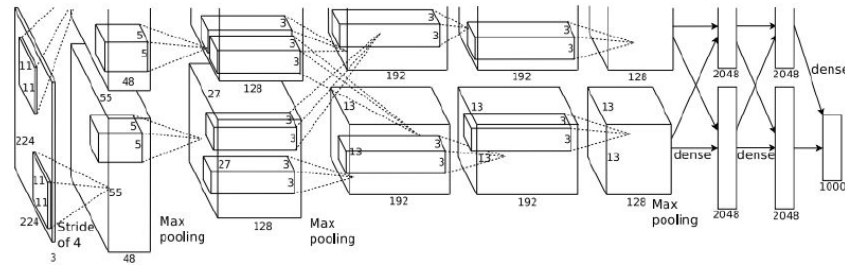
Q: what is the total number of parameters in this layer?

$$\frac{227-11}{4} + 1 = 55$$

CNN

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3

First layer (CONV1): 96 11x11 filters applied at stride 4

=>

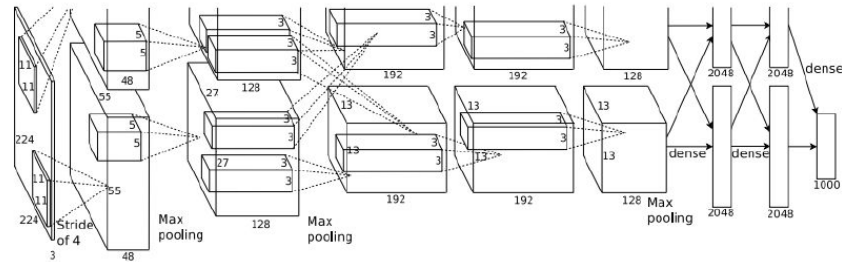
Output volume **[55x55x96]**

Parameters: $(11*11*3)*96 = \mathbf{35K}$

CNN

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 image

After CONV1: 55x55x96

Second layer (POOL1): 3x3 filters applied at stride 2

Output volume: 27x27x96

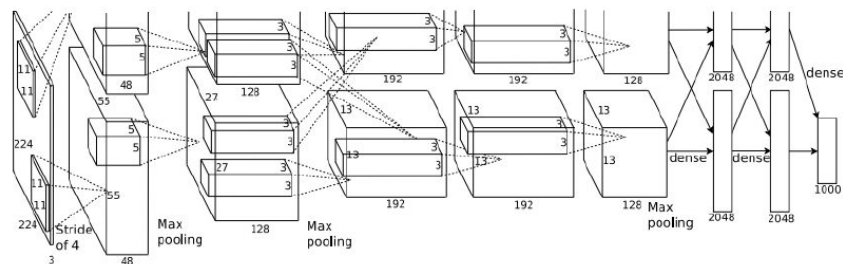
$$\frac{55-3}{2} + 1 = 27$$

Q: what is the number of parameters in this layer?

CNN

Case Study: AlexNet

[Krizhevsky et al. 2012]



Input: 227x227x3 image

After CONV1: 55x55x96

Second layer (POOL1): 3x3 filters applied at stride 2

Output volume: 27x27x96

Parameters: 0!

假设 pooling 层采用的是 max pooling, 那么实际上这里根本用不到参数, 只需要依赖 max() 方法就可以, 所以没有权重参数

CNN

Case Study: AlexNet

[Krizhevsky et al. 2012]

Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

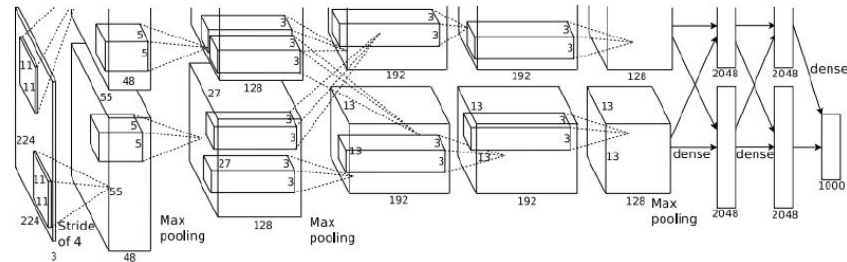
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)



CNN

Case Study: AlexNet

[Krizhevsky et al. 2012]

Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

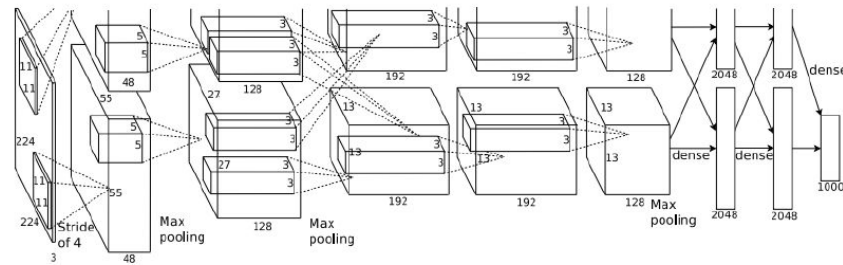
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)



Details/Retrospectives:

- First use of ReLU
- Used Norm layers (not common anymore)
- Heavy data augmentation
- Dropout 0.5
- Batch size 128
- SGD Momentum 0.9
- Learning rate 1e-2, reduced by 10 Manually when val accuracy plateaus
- L2 weight decay 5e-4
- 7 CNN ensemble: 18.2% -> 15.4%

CNN

1. 最大的改变: CONV1 (第一层卷积)

AlexNet 的做法 (旧): 11x11 的超大滤波器, stride 4 (步长4)。

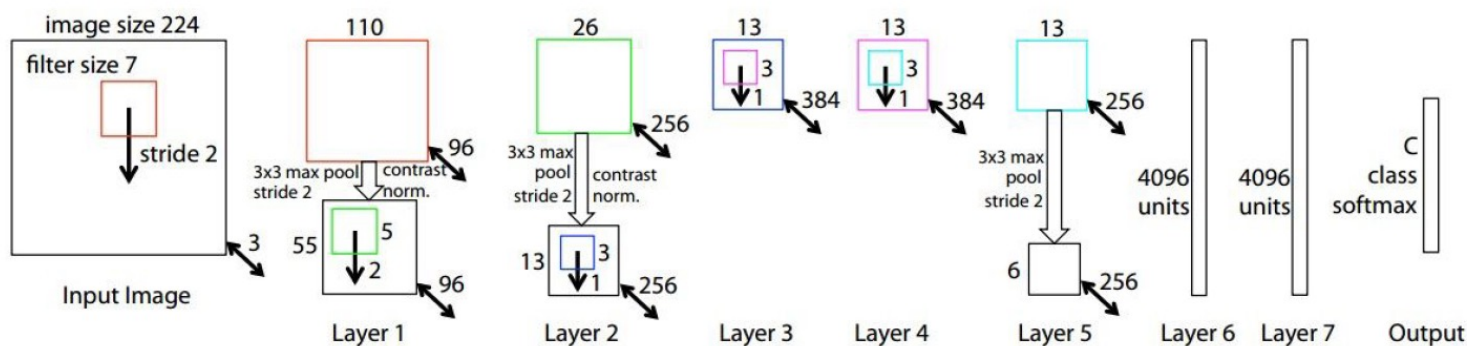
缺点: 这太“激进”了! 11x11 的大“网”和 4 格一“跳”的步长, 导致它在第一层就错过了很多图片中的细节和小特征。

ZFNet 的做法 (新): 7x7 的滤波器, stride 2 (步长2)。

优点: 如图表左上角所示, 更小的滤波器 (filter size 7) 和更小的步长 (stride 2), 意味着网络在第一层可以更“仔细”、更“温柔”地扫描图片。它能捕捉到那些被 AlexNet 忽略的细节, 为后续的特征提取打下更好的基础。

结果: 你可以看到, 经过第一层卷积后, AlexNet 的特征图很小 (55x55), 而 ZFNet 的特征图更大 (110x110), 保留了更多的空间信息。

Case Study: ZFNet [Zeiler and Fergus, 2013]



AlexNet but:

CONV1: change from (11x11 stride 4) to (7x7 stride 2)

CONV3,4,5: instead of 384, 384, 256 filters use 512, 1024, 512

ImageNet top 5 error: 15.4% -> 14.8%

2. 另一个改变: 更“深”的中间层

AlexNet 的做法 (旧): 中间的 CONV3, 4, 5 层的滤波器数量分别是 384, 384, 256。

ZFNet 的做法 (新): 把这些层的滤波器数量大幅增加到了 512, 1024, 512。

优点: 我们之前学过, 滤波器的数量 (K) 决定了输出的“深度” (D_2)。通过增加 K , ZFNet 给了网络更强大的“特征容量”, 让它可以在中间层学习到更丰富、更复杂的特征组合。

"Only 3x3 CONV stride 1, pad 1" (只用 3x3 卷积)

3x3: 这是能捕捉“上/下/左/右/中”概念的最小滤波器尺寸。

stride 1: 步长为1, 保证“仔仔细细”地扫描。

pad 1: 这是天才之举! 我们来算一下黄金公式 $(W - F + 2P) / S + 1$:

$$(W - 3 + 2*1) / 1 + 1 = W - 3 + 2 + 1 = W$$

这意味着, 只要经过 3x3, pad 1 的卷积, 特征图的空间尺寸 (W, H) 永远不会改变!

这有什么用? 这让 VGGNet 可以随心所欲地“堆叠”卷积层 (比如连堆 3 个 conv3-256), 而不用担心图片尺寸在中间被“意外”缩小。

CNN

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Only 3x3 CONV stride 1, pad1
And 2x2 MAX POOL stride 2

best model

11.2% top 5 error in ILSVRC 2013

->

7.3% top 5 error

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: Number of parameters (in millions).

Network	A, A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

"And 2x2 MAX POOL stride 2" (只用 2x2 池化)

VGGNet 只在它“想”缩小尺寸的时候, 才显式地插入一个 2x2, stride 2 的池化层。

我们再算一次黄金公式 $(W - F) / S + 1$:

$$(W - 2) / 2 + 1 = W/2 - 1 + 1 = W/2$$

这意味着, 每一次池化, 特征图的长宽都“干净利落”地缩小一半。224 -> 112 -> 56 -> 28 -> 14 -> 7。

CNN

INPUT: [224x224x3] memory: 224*224*3=150K params: 0 (not counting biases)

CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*3)*64 = 1,728

CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*64)*64 = 36,864

POOL2: [112x112x64] memory: 112*112*64=800K params: 0

CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*64)*128 = 73,728

CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*128)*128 = 147,456

POOL2: [56x56x128] memory: 56*56*128=400K params: 0

CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*128)*256 = 294,912

CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824

CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824

POOL2: [28x28x256] memory: 28*28*256=200K params: 0

CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*256)*512 = 1,179,648

CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296

CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296

POOL2: [14x14x512] memory: 14*14*512=100K params: 0

CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296

CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296

CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296

POOL2: [7x7x512] memory: 7*7*512=25K params: 0

FC: [1x1x4096] memory: 4096 params: 7*7*512*4096 = 102,760,448

FC: [1x1x4096] memory: 4096 params: 4096*4096 = 16,777,216

FC: [1x1x1000] memory: 1000 params: 4096*1000 = 4,096,000

ConvNet Configuration			
B	C	D	
13 weight layers	16 weight layers	16 weight layers	19
put (224 × 224 RGB image)			
conv3-64	conv3-64	conv3-64	cc
conv3-64	conv3-64	conv3-64	cc
maxpool			
conv3-128	conv3-128	conv3-128	co
conv3-128	conv3-128	conv3-128	co
maxpool			
conv3-256	conv3-256	conv3-256	co
conv3-256	conv3-256	conv3-256	co
	conv1-256	conv3-256	co
maxpool			
conv3-512	conv3-512	conv3-512	co
conv3-512	conv3-512	conv3-512	co
	conv1-512	conv3-512	co
maxpool			
conv3-512	conv3-512	conv3-512	co
conv3-512	conv3-512	conv3-512	co
	conv1-512	conv3-512	co
maxpool			
FC-4096			
FC-4096			
FC-1000			
soft-max			

CNN

INPUT: [224x224x3] memory: 224*224*3=150K params: 0 (not counting biases)
 CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*3)*64 = 1,728
 CONV3-64: [224x224x64] memory: 224*224*64=3.2M params: (3*3*64)*64 = 36,864
 POOL2: [112x112x64] memory: 112*112*64=800K params: 0
 CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*64)*128 = 73,728
 CONV3-128: [112x112x128] memory: 112*112*128=1.6M params: (3*3*128)*128 = 147,456
 POOL2: [56x56x128] memory: 56*56*128=400K params: 0
 CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*128)*256 = 294,912
 CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
 CONV3-256: [56x56x256] memory: 56*56*256=800K params: (3*3*256)*256 = 589,824
 POOL2: [28x28x256] memory: 28*28*256=200K params: 0
 CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*256)*512 = 1,179,648
 CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
 CONV3-512: [28x28x512] memory: 28*28*512=400K params: (3*3*512)*512 = 2,359,296
 POOL2: [14x14x512] memory: 14*14*512=100K params: 0
 CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
 CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
 CONV3-512: [14x14x512] memory: 14*14*512=100K params: (3*3*512)*512 = 2,359,296
 POOL2: [7x7x512] memory: 7*7*512=25K params: 0
 FC: [1x1x4096] memory: 4096 params: 7*7*512*4096 = 102,760,448
 FC: [1x1x4096] memory: 4096 params: 4096*4096 = 16,777,216
 FC: [1x1x1000] memory: 1000 params: 4096*1000 = 4,096,000

TOTAL memory: 24M * 4 bytes ~= 93MB / image (only forward! ~*2 for bwd)

TOTAL params: 138M parameters

ConvNet Configuration			
B	C	D	
13 weight layers	16 weight layers	16 weight layers	19
put (224 × 224 RGB image)			
conv3-64	conv3-64	conv3-64	cc
conv3-64	conv3-64	conv3-64	cc
maxpool			
conv3-128	conv3-128	conv3-128	coi
conv3-128	conv3-128	conv3-128	coi
maxpool			
conv3-256	conv3-256	conv3-256	coi
conv3-256	conv3-256	conv3-256	coi
	conv1-256	conv3-256	coi
maxpool			
conv3-512	conv3-512	conv3-512	coi
conv3-512	conv3-512	conv3-512	coi
	conv1-512	conv3-512	coi
maxpool			
conv3-512	conv3-512	conv3-512	coi
conv3-512	conv3-512	conv3-512	coi
	conv1-512	conv3-512	coi
maxpool			
FC-4096			
FC-4096			
FC-1000			
soft-max			

CNN

"Note: Most memory is in early CONV" (大部分“内存”消耗在“早期的”卷积层)

看“memory”列：CONV3-64 层的激活图 (activations) 尺寸是 $224 \times 224 \times 64$ 。这个方块本身就要占用 3.2M 的内存（这还只是一个开始！）。

为什么？因为 VGG 为了保持尺寸不变，在前面几层一直维持着 224×224 和 112×112 这样超大的空间分辨率。这些巨大的“中间产物”（激活图）在训练时必须全部保存在显存中，导致内存开销极其恐怖。

INPUT: $[224 \times 224 \times 3]$ memory: $224 \times 224 \times 3 = 150K$ params: 0 (not counting biases)
CONV3-64: $[224 \times 224 \times 64]$ memory: $224 \times 224 \times 64 = 3.2M$ params: $(3 \times 3 \times 64) = 1,728$
CONV3-64: $[224 \times 224 \times 64]$ memory: $224 \times 224 \times 64 = 3.2M$ params: $(3 \times 3 \times 64) = 36,864$
POOL2: $[112 \times 112 \times 64]$ memory: $112 \times 112 \times 64 = 800K$ params: 0
CONV3-128: $[112 \times 112 \times 128]$ memory: $112 \times 112 \times 128 = 1.6M$ params: $(3 \times 3 \times 64) \times 128 = 73,728$
CONV3-128: $[112 \times 112 \times 128]$ memory: $112 \times 112 \times 128 = 1.6M$ params: $(3 \times 3 \times 128) \times 128 = 147,456$
POOL2: $[56 \times 56 \times 128]$ memory: $56 \times 56 \times 128 = 400K$ params: 0
CONV3-256: $[56 \times 56 \times 256]$ memory: $56 \times 56 \times 256 = 800K$ params: $(3 \times 3 \times 128) \times 256 = 294,912$
CONV3-256: $[56 \times 56 \times 256]$ memory: $56 \times 56 \times 256 = 800K$ params: $(3 \times 3 \times 256) \times 256 = 589,824$
CONV3-256: $[56 \times 56 \times 256]$ memory: $56 \times 56 \times 256 = 800K$ params: $(3 \times 3 \times 256) \times 256 = 589,824$
POOL2: $[28 \times 28 \times 256]$ memory: $28 \times 28 \times 256 = 200K$ params: 0
CONV3-512: $[28 \times 28 \times 512]$ memory: $28 \times 28 \times 512 = 400K$ params: $(3 \times 3 \times 256) \times 512 = 1,179,648$
CONV3-512: $[28 \times 28 \times 512]$ memory: $28 \times 28 \times 512 = 400K$ params: $(3 \times 3 \times 512) \times 512 = 2,359,296$
CONV3-512: $[28 \times 28 \times 512]$ memory: $28 \times 28 \times 512 = 400K$ params: $(3 \times 3 \times 512) \times 512 = 2,359,296$
POOL2: $[14 \times 14 \times 512]$ memory: $14 \times 14 \times 512 = 100K$ params: 0
CONV3-512: $[14 \times 14 \times 512]$ memory: $14 \times 14 \times 512 = 100K$ params: $(3 \times 3 \times 512) \times 512 = 2,359,296$
CONV3-512: $[14 \times 14 \times 512]$ memory: $14 \times 14 \times 512 = 100K$ params: $(3 \times 3 \times 512) \times 512 = 2,359,296$
CONV3-512: $[14 \times 14 \times 512]$ memory: $14 \times 14 \times 512 = 100K$ params: $(3 \times 3 \times 512) \times 512 = 2,359,296$
POOL2: $[7 \times 7 \times 512]$ memory: $7 \times 7 \times 512 = 25K$ params: 0
FC: $[1 \times 1 \times 4096]$ memory: 4096 params: $7 \times 7 \times 512 \times 4096 = 102,760,448$
FC: $[1 \times 1 \times 4096]$ memory: 4096 params: $4096 \times 4096 = 16,777,216$
FC: $[1 \times 1 \times 1000]$ memory: 1000 params: $4096 \times 1000 = 4,096,000$

TOTAL memory: $24M \times 4 \text{ bytes} \approx 93MB$ / image (only forward! ~ 2 for bwd)

TOTAL params: 138M parameters

Note:

Most memory is in early CONV

Most params are in late FC

"Note: Most params are in late FC" (大部分“参数”消耗在“晚期的”全连接层)

看“params”列：所有的 CONV 层（由于 3×3 很小，且参数共享）加起来的参数都还好。

但是，看第一个 FC 层！

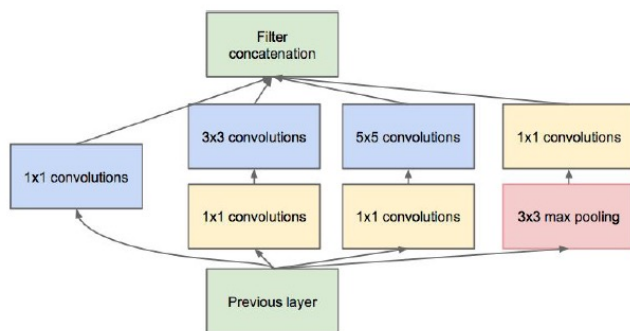
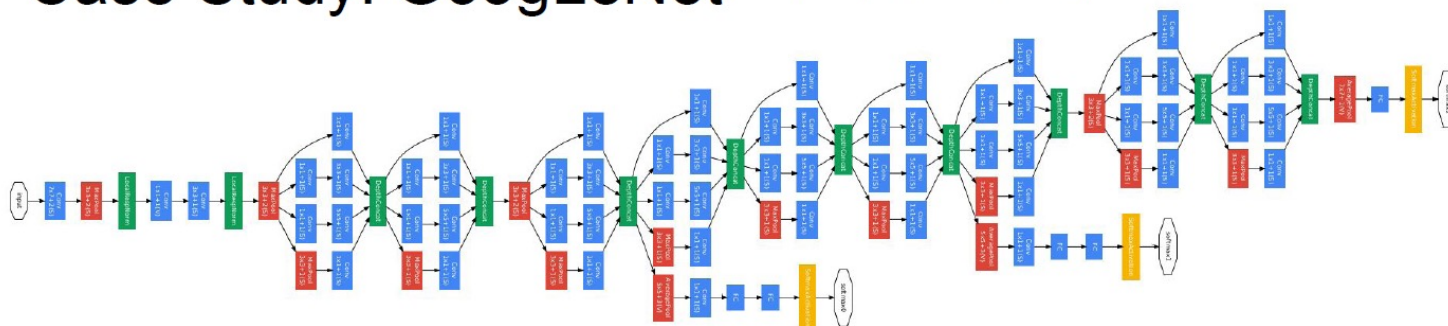
它的输入是 $7 \times 7 \times 512$ （来自最后一个 POOL 层），输出是 4096。

我们学的“神经元”知识派上用场了：它需要 $(7 \times 7 \times 512) \times 4096 = 102,760,448$ （超过 1 亿）个权重！

结果：TOTAL params: 138M (1.38 亿)。VGGNet 是一个**极其“臃肿”和“沉重”**的庞然大物。它的参数量几乎是 AlexNet 的 10 倍！

CNN

Case Study: GoogLeNet [Szegedy et al., 2014]



Inception module

ILSVRC 2014 winner (6.7% top 5 error)

CNN

Case Study: GoogLeNet

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

Fun features:

- Only 5 million params!
(Removes FC layers completely)

Compared to AlexNet:

- 12X less params
- 2x more compute
- 6.67% (vs. 16.4%)

12x less params (比 AlexNet 少 12 倍参数)

2x more compute (但计算量更密集)

6.67% (vs. 16.4%) (错误率完胜!)

Case Study: ResNet [He et al., 2015] ILSVRC 2015 winner (3.6% top 5 error)



MSRA @ ILSVRC & COCO 2015 Competitions

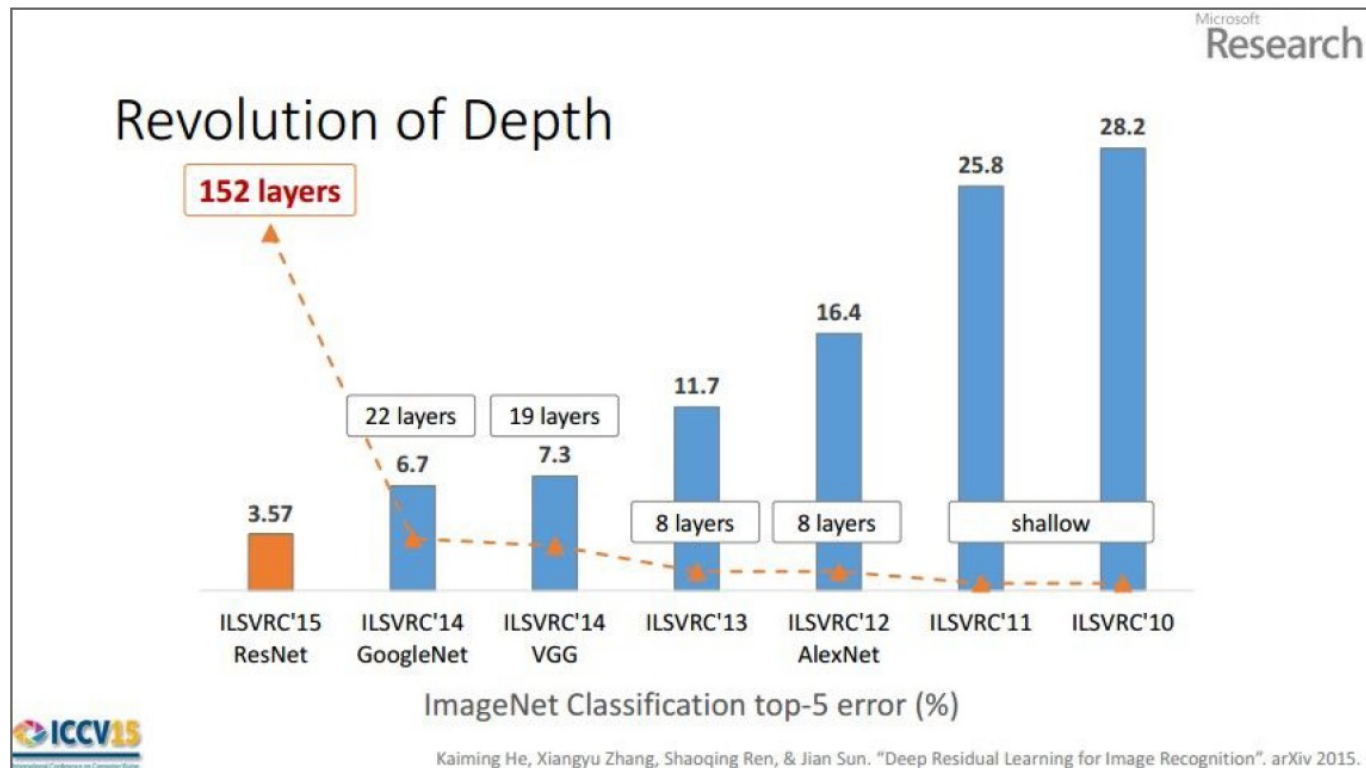
- **1st places** in all five main tracks
 - ImageNet Classification: “Ultra-deep” (quote Yann) **152-layer** nets
 - ImageNet Detection: **16%** better than 2nd
 - ImageNet Localization: **27%** better than 2nd
 - COCO Detection: **11%** better than 2nd
 - COCO Segmentation: **12%** better than 2nd

*improvements are relative numbers

Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. “Deep Residual Learning for Image Recognition”. arXiv 2015.

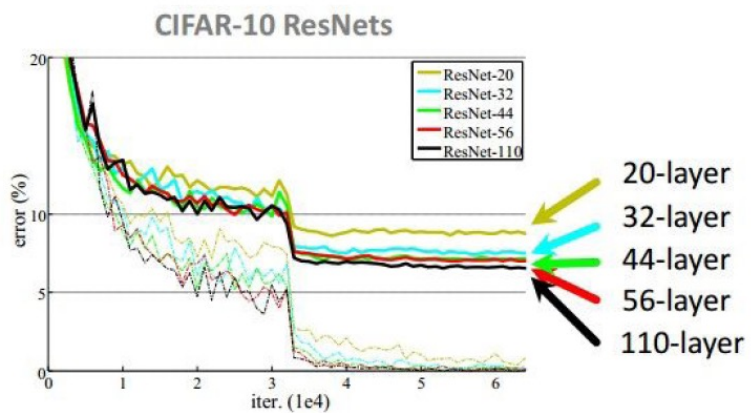
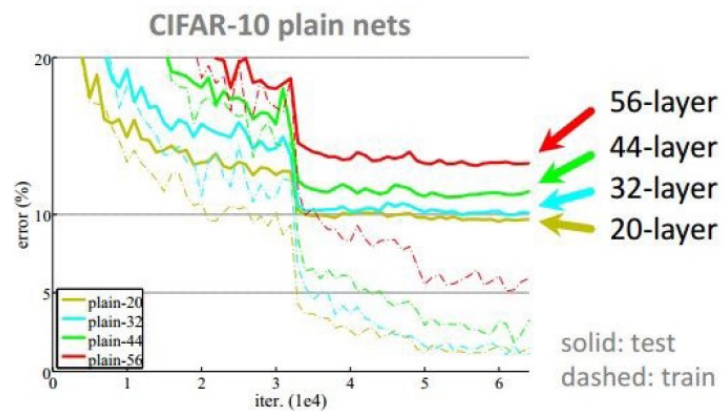


CNN



CNN

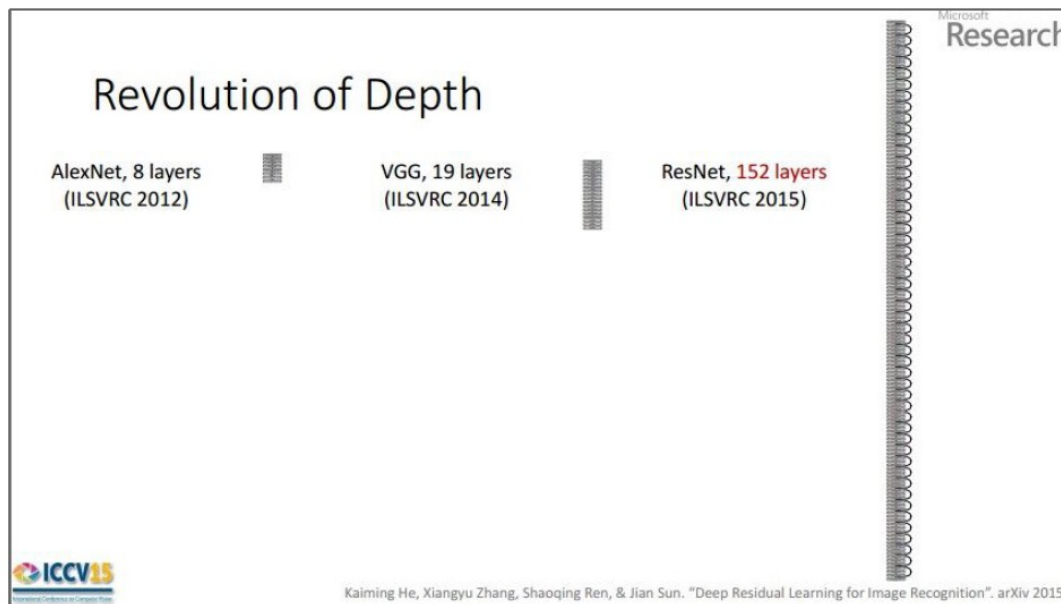
CIFAR-10 experiments



CNN

Case Study: ResNet [He et al., 2015]

ILSVRC 2015 winner (3.6% top 5 error)



2-3 weeks of training on 8 GPU machine

at runtime: faster than a VGGNet! (even though it has 8x more layers)

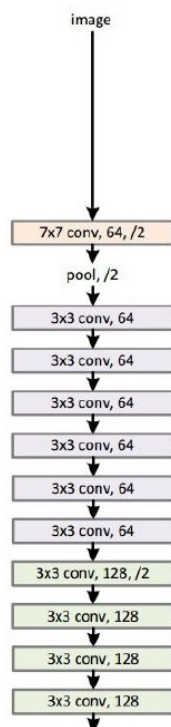
(slide from Kaiming He's recent presentation)

CNN

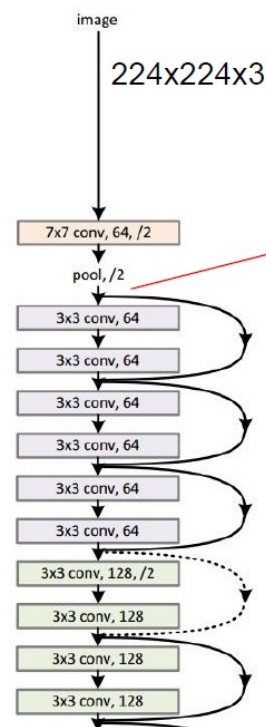
Case Study: ResNet

[He et al., 2015]

34-layer plain



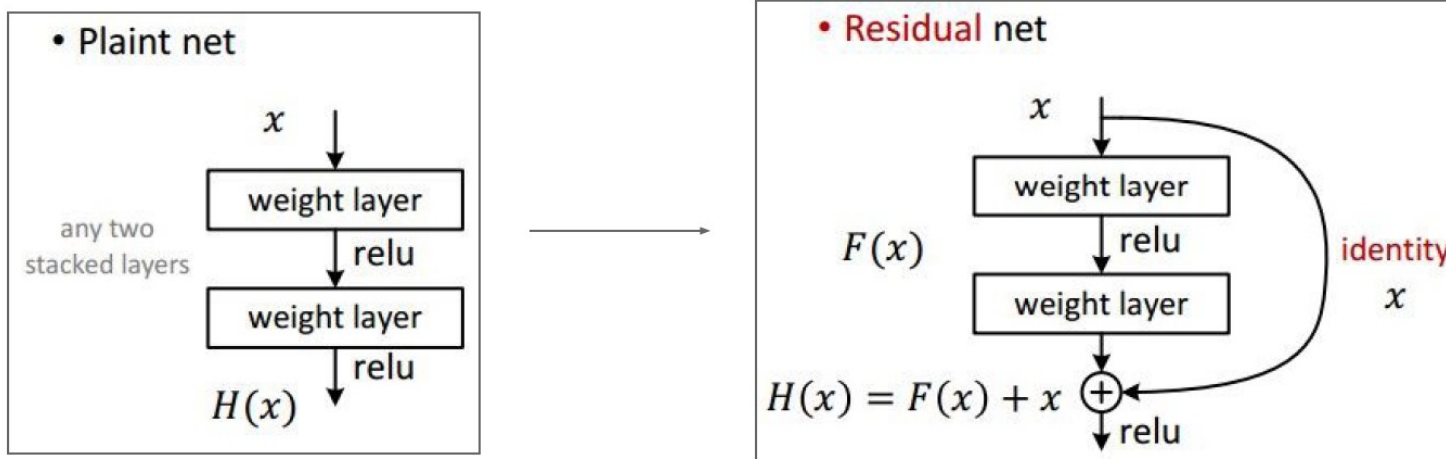
34-layer residual



spatial dimension
only 56x56!

CNN

Case Study: ResNet [He et al., 2015]



CNN

1. "ConvNets stack CONV, POOL, FC layers"

(CNN 就是 CONV, POOL, FC 层的堆叠)

我们的认证：正确！我们称它们为“三剑客” (CONV, ReLU, POOL) 和“决策者”(FC)。这就是经典 CNN 架构的“骨架”，我们在 AlexNet 和 VGGNet 中都看到了这个清晰的结构。

Summary

- ConvNets stack CONV, POOL, FC layers
- Trend towards smaller filters and deeper architectures
- Trend towards getting rid of POOL/FC layers (just CONV)
- Typical architectures look like

$[(\text{CONV-RELU}) * N - \text{POOL}] * M - (\text{FC-RELU}) * K, \text{SOFTMAX}$

where N is usually up to ~ 5 , M is large, $0 \leq K \leq 2$.

- but recent advances such as ResNet/GoogLeNet challenge this paradigm

2. "Trend towards smaller filters and deeper architectures"

(趋势是使用更小的滤波器和更深的网络架构)

我们的认证：太对了！这正是我们看到的 VGGNet (2014) 对 AlexNet (2012) 和 ZFNet (2013) 的“革命”。

AlexNet 用了“激进”的 11×11 滤波器。

ZFNet 改良为 7×7 。

VGGNet 则定下了“极简主义”的规矩：只用 3×3 ！

为了弥补小滤波器的“视野”问题，VGGNet 把网络堆叠到了史无前例的 16-19 层 (deeper architectures)，并证明了“深度”是王道。

4. "Typical architectures look like..."

(经典架构的“配方”)

$[(\text{CONV-RELU}) * N - \text{POOL}] * M - (\text{FC-RELU}) * K, \text{SOFTMAX}$

我们的认证：这是一个对 AlexNet 和 VGGNet 的完美“公式”总结！

$[(\text{CONV-RELU}) * N - \text{POOL}]$: 这就是一个“模块”。比如 VGGNet 里 CONV-RELU 堆叠 $N=2$ 或 3 次，然后跟上一个 POOL 层。

*M: 整个网络就是把这个“模块”重复 M 次 (比如 5 次)。

$(\text{FC-RELU}) * K, \text{SOFTMAX}$: 最后，接上 $K=2$ 或 3 个全连接层，并用 SOFTMAX 输出分类。

5. "...but recent advances such as ResNet/GoogLeNet challenge this paradigm"

(但是，像 ResNet/GoogLeNet 这样的进展正在挑战这个“经典配方”)

我们的认证：绝对正确！

GoogLeNet (我们刚看过)：它用“Inception 模块” ($1 \times 1, 3 \times 3, 5 \times 5$ 并行计算) 和“全局平均池化”，彻底打破了上面那个“线性堆叠”的“经典配方”。

ResNet (幻灯片提到的新网络)：它引入了“残差连接” (跳跃连接)，允许网络盖得更深 (几百上千层)，这是对这个“经典配方”的又一次颠覆。

3. "Trend towards getting rid of POOL/FC layers (just CONV)"

(趋势是抛弃 POOL/FC 层，(更多地) 只用 CONV)

我们的认证：我们亲眼见证了！

抛弃 FC 层：这正是 GoogLeNet (2014) 的“神来之笔”！它用“全局平均池化 (Global Average Pooling)”替换掉了 VGGNet 里那“臃肿”的 1 亿参数的 FC 层，让模型参数从 138M 锐减到 5M！

抛弃 POOL 层：这个趋势 (比如用 $\text{stride}=2$ 的卷积层来代替池化层) 在这几张幻灯片里尚未明确体现，但它预示了更现代的网络设计。